

DOSSIER DE PROJET

Hesias Fleet

Solution de gestion de flotte de véhicules

Web · Desktop · Mobile

Major Project — B3 Développement

Hesias — Hautes études en informatique

Année 2025 – 2026

Aymeric Matos

Sommaire

Sommaire	2
1. Introduction.....	5
1.1. Résumé du projet (English summary)	6
2. Compétences du référentiel mises en œuvre	7
2.1. Activité 1 — Concevoir et développer des composants d'interface utilisateur en intégrant les recommandations de sécurité	7
2.2. Activité 2 — Concevoir et développer la persistance des données en intégrant les recommandations de sécurité	7
2.3. Activité 3 — Concevoir et développer une application multicouche répartie en intégrant les recommandations de sécurité	8
3. Expression des besoins	9
3.1. Objectifs fonctionnels	9
3.2. Limites et périmètre	10
3.3. Contraintes	10
3.4. Livrables attendus	11
4. Spécifications fonctionnelles	12
4.1. Règles de gestion — authentification et utilisateurs	12
4.2. Règles de gestion — magasin.....	12
4.3. Règles de gestion — véhicules et journal	12
4.4. Cas d'utilisation principaux	13
5. Gestion de projet : comment s'est déroulé le projet ?	14
5.1. Démarche générale	14
5.2. Planning et suivi	14
5.3. Objectifs de qualité	15
5.4. Difficultés rencontrées et arbitrages	16
5.5. Méthode de travail et rythme.....	16
5.6. Gestion des risques	17
5.7. Estimation et répartition de la charge	17
6. Maquettage et identité visuelle	18
6.1. La démarche de maquettage	18
6.2. L'identité visuelle	18
6.3. Du papier à l'écran : ce que les maquettes n'avaient pas prévu	18
7. Environnement de travail.....	20
8. Environnement technique.....	21
8.1. Le backend et sa base de données	21
8.2. Le frontend web et le desktop	21
8.3. Le mobile Android	21
9. Langages et technologies : choix et avantages	23
9.1. C# et ASP.NET Core 8 pour le backend	23
9.2. Entity Framework Core et PostgreSQL 16 pour les données.....	23
9.3. Angular 19 et Angular Material pour le web	23
9.4. Electron 43 pour le desktop	24
9.5. Kotlin et Android natif pour le mobile	24
9.6. Docker pour la conteneurisation	24
9.7. Les alternatives étudiées.....	24
10. Architecture de l'application	26
10.1. Un backend en trois couches	26
10.2. Un front décliné en deux supports	27
10.3. Un mobile natif consommant la même API	27

10.4. Cheminement d'une requête, de bout en bout	27
10.5. Pourquoi REST, et pas autre chose ?	27
11. La base de données	29
11.1. Lecture guidée du modèle	30
11.2. Choix de conception notables	30
11.3. La migration : le schéma versionné avec le code	31
11.4. Le dictionnaire de données, en bref	31
11.5. Volumétrie et comportement attendu	32
12. L'API REST	33
12.1. Principes	33
12.2. L'authentification par JWT	33
12.3. Le tour des endpoints, contrôler par contrôler	34
12.4. Les DTOs : le contrat explicite de l'API	34
12.5. Conventions de réponse et gestion des erreurs	35
12.6. Deux échanges commentés	35
13. Le frontend web en détail	37
13.1. Organisation du code	37
13.2. La gestion d'état avec les signals	37
13.3. La protection des routes et la gestion du jeton	37
13.4. La déclinaison desktop avec Electron	38
13.5. Les formulaires réactifs et la validation	38
13.6. L'identité visuelle dans le code	38
14. L'application mobile en détail	40
14.1. Architecture MVVM	40
14.2. Le contrat REST typé avec Retrofit	40
14.3. Le journal unifié	41
14.4. L'identification par QR code	41
14.5. La saisie minimale	41
14.6. La gestion de session	41
15. Organisation de l'interface : pourquoi ces écrans ?	42
15.1. La structure générale : une sidebar à trois entrées	42
15.2. Le Dashboard comme écran d'accueil	42
15.3. La fiche véhicule en trois onglets	42
15.4. Des popups plutôt que des pages pour la saisie	43
15.5. Le magasin : autocomplétion et bouton d'édition	43
15.6. L'application mobile : le parcours du technicien	43
16. Le fonctionnement du site	44
16.1. Le parcours type	44
16.2. La prise en main par profil	45
16.3. Les cas particuliers et les états d'erreur	45
17. Réalisations significatives (extraits de code)	46
17.1. Le déstockage FIFO par prix (StockService)	46
17.2. Le calcul des butées (DeadlineService)	47
17.3. L'orchestration métier : la création d'une opération	47
17.4. Les intercepteurs : l'authentification traitée une seule fois	48
17.5. Le stockage chiffré du jeton sur mobile	49
18. Sécurité	50
18.1. Les mesures mises en œuvre	50
18.2. La veille sur les vulnérabilités de sécurité	50
18.3. Une situation de travail ayant nécessité une recherche	50
18.4. Le projet au prisme de l'OWASP Top 10	51
19. Plan de tests et jeu d'essai	52

19.1. La stratégie de test.....	52
19.2. Le jeu d'essai : la consommation FIFO du stock.....	53
19.3. Analyse des écarts et non-régression	53
19.4. Le jeu de données de test	53
20. Déploiement.....	55
20.1. L'installation sur un poste vierge	55
20.2. Vers la production : ce qui changerait	55
21. Axes d'amélioration.....	57
21.1. L'ajout de fonctionnalités	57
21.2. La sécurité et l'optimisation	57
21.3. Le déploiement	57
21.4. La feuille de route priorisée	58
22. Conclusion	59
23. Glossaire	60

1. Introduction

Hesias Fleet est le Major Project de ma troisième année de Bachelor Développement à Hesias. Le sujet, proposé par l'éditeur fictif Quantum Quotient Labs, consistait à concevoir et développer une solution complète de gestion de flotte de véhicules à destination des entreprises. L'objectif métier est simple à énoncer mais riche fonctionnellement : centraliser les opérations de maintenance effectuées sur les véhicules d'une flotte, suivre le stock de consommables associés (filtres, plaquettes de frein, huile...) et gérer la périodicité des opérations récurrentes grâce à un système de butées calendaires et kilométriques.

Pour comprendre l'intérêt d'un tel outil, il faut se placer du point de vue d'une entreprise qui exploite des véhicules au quotidien : une société de BTP avec ses fourgons et ses engins de chantier, une collectivité avec ses véhicules de service, un artisan avec ses utilitaires. Dans la plupart de ces structures, le suivi de l'entretien repose encore sur des carnets papier, des tableurs partagés ou, pire, sur la mémoire des uns et des autres. Les conséquences sont connues : des vidanges oubliées qui raccourcissent la vie des moteurs, des contrôles réglementaires dépassés, des pièces commandées en urgence au prix fort parce que personne n'avait vu que le stock de filtres était épuisé, et une incapacité totale à répondre à des questions pourtant simples — combien a coûté l'entretien de ce véhicule cette année ? Qui a sorti ces plaquettes de frein du magasin, et quand ?

Hesias Fleet répond précisément à ces questions du quotidien : quand la prochaine vidange de tel véhicule doit-elle être faite ? Reste-t-il des filtres à huile en stock, et à quel prix de revient ? Quelles opérations arrivent à échéance dans les prochaines semaines ? L'application centralise le journal d'entretien de chaque véhicule, le stock du magasin de pièces, et alerte automatiquement les utilisateurs lorsqu'une opération récurrente arrive à échéance, que cette échéance soit exprimée en temps (« tous les deux ans ») ou en distance parcourue (« tous les 20 000 kilomètres »).

Le cahier des charges imposait une déclinaison de la solution sur trois supports. D'abord une **application web**, version complète de gestion, qui doit obligatoirement s'exécuter dans un container Docker. Ensuite une **application desktop**, sur la plateforme de mon choix entre Windows, macOS et Linux, elle aussi version complète. Enfin une **application mobile**, iOS ou Android au choix, pensée comme un outil de saisie sur site pour les techniciens pendant les opérations d'entretien : consultation du journal d'un véhicule et saisie rapide des opérations effectuées, en temps quasi réel, directement depuis l'atelier.

Le choix des technologies était entièrement libre — une liberté qui est aussi une responsabilité, puisqu'il fallait la justifier. J'ai retenu une architecture centrée sur une **API REST en C# / ASP.NET Core 8** adossée à une base **PostgreSQL 16** conteneurisée, un frontend web **Angular 19** avec Angular Material, une application desktop **Electron** qui réutilise intégralement le build Angular, et une application **Android native en Kotlin**. Ces choix, leurs avantages et les alternatives que j'ai écartées sont détaillés dans la section 9 de ce dossier.

Ce projet compte également pour le titre Bac+4 « Concepteur Développeur d'Applications » : le présent dossier suit donc le plan type du référentiel de validation du titre. Il présente successivement les compétences couvertes, l'expression des besoins et les spécifications, la gestion du projet et son déroulement, les environnements de travail et techniques, l'architecture, la base de données et l'API, les interfaces des trois clients et leur organisation, les réalisations les plus significatives avec leurs

extraits de code argumentés, la sécurité, le plan de tests et son jeu d'essai, le déploiement, et enfin les axes d'amélioration identifiés pour la suite.

1.1. Résumé du projet (English summary)

Hesias Fleet is a full-stack fleet-management solution developed as my final-year Major Project at Hesias. It targets companies that operate a fleet of vehicles and need to centralise maintenance operations, track the stock of consumables used during servicing, and manage the recurrence of periodic operations. The solution is delivered on three platforms: a complete web application, a Windows desktop application, and an Android mobile application used by technicians for on-site data entry.

The backend is a REST API built with C# and ASP.NET Core 8, organised in three layers (Core, Infrastructure, API) following clean-architecture principles. Data is persisted in a PostgreSQL 16 database running in a Docker container, accessed through Entity Framework Core with a code-first migration. Authentication relies on JSON Web Tokens signed with HMAC-SHA256, and passwords are hashed with BCrypt. The web frontend is an Angular 19 single-page application styled with Angular Material; it is packaged by a multi-stage Dockerfile and served by nginx, which also proxies API calls to avoid CORS issues in production. The desktop application is built with Electron and reuses the Angular build. The mobile application is written in Kotlin with Retrofit and coroutines, stores the JWT in encrypted preferences, and can identify a vehicle by scanning a QR code.

Key features include a store module with FIFO stock consumption (cheapest batches first), a per-vehicle logbook mixing operations, meta-operations and notes, deadline tracking by date and mileage with in-app notifications, and input assistance such as autocompletion and smart default values.

2. Compétences du référentiel mises en œuvre

Le sujet Hesias Fleet a été conçu pour couvrir l'ensemble des activités types du référentiel du titre « Concepteur Développeur d'Applications ». Plutôt qu'une simple liste, cette section explique, activité par activité et compétence par compétence, où et comment chacune a été réellement mise en œuvre dans le projet — les sections citées entre parenthèses renvoient aux endroits du dossier où la réalisation correspondante est détaillée.

2.1. Activité 1 — Concevoir et développer des composants d'interface utilisateur en intégrant les recommandations de sécurité

La compétence « **Maquetter une application** » a été exercée en tout début de projet : conformément au cahier des charges, l'ensemble des écrans des trois supports a été maqueté avant la première ligne de code — écran de connexion, tableau de bord, liste et fiche des véhicules avec leurs onglets, magasin et fenêtres de saisie pour le web et le desktop ; parcours complet du technicien pour le mobile. La section 6 décrit cette démarche et ce qu'elle a apporté à la conception.

La compétence « **Développer des composants d'accès aux données** » correspond à la couche Infrastructure du backend : des repositories C# construits sur Entity Framework Core 8 et le provider Npgsql encapsulent tous les accès à PostgreSQL derrière des interfaces du domaine, si bien qu'aucun controller ne manipule directement la base (sections 10 et 11). La compétence « **Développer la partie front-end d'une interface utilisateur web** » est incarnée par la SPA Angular 19 : composants standalone chargés à la demande, bibliothèque Angular Material, gestion d'état par signals, formulaires réactifs validés, intercepteur HTTP et guard de route (section 13). La compétence « **Développer la partie back-end d'une interface utilisateur web** » recouvre l'API REST ASP.NET Core 8 elle-même : controllers minces, DTOs dédiés, injection de dépendances, authentification JWT et documentation Swagger (section 12).

Les recommandations de sécurité traversent ces quatre compétences plutôt qu'elles ne s'y ajoutent : validation systématique côté serveur, aucune donnée sensible exposée par l'API, jeton d'authentification porté par en-tête et jamais placé dans les URL, et messages d'erreur qui ne révèlent rien d'exploitable. La section 18 leur est entièrement consacrée.

2.2. Activité 2 — Concevoir et développer la persistance des données en intégrant les recommandations de sécurité

La compétence « **Concevoir une base de données** » s'est exercée dans le passage de l'expression des besoins au modèle relationnel de onze tables présenté en section 11 : identifier les entités (véhicules, opérations, pièces, lots de stock, mouvements, notes, utilisateurs), leurs relations et leurs cardinalités, et surtout faire les choix structurants — comme la modélisation du stock en lots datés et valorisés, sans laquelle la règle de déstockage FIFO par prix du sujet serait inapplicable. La compétence « **Mettre en place une base de données** » correspond au déploiement concret de PostgreSQL 16 dans un container Docker décrit par docker-compose, avec son volume persistant, et à la création du schéma par la migration EF Core initiale, rejouable sur n'importe quel poste.

La compétence « **Développer des composants dans le langage d'une base de données** » se retrouve dans tout ce que la migration et le DbContext définissent au niveau de la base elle-même : contraintes d'intégrité référentielle, index d'unicité sur l'email et le login des utilisateurs, précision

des types monétaires, et comportements de suppression choisis selon le sens métier — suppression en cascade lorsqu'un véhicule disparaît avec son journal, mais détachement (SET NULL) lorsqu'une méta-opération est supprimée, pour ne pas perdre les opérations qu'elle regroupait. Côté sécurité, les mots de passe ne sont persistés que hachés par BCrypt, et toutes les requêtes générées par EF Core sont paramétrées, ce qui écarte par construction l'injection SQL.

2.3. Activité 3 — Concevoir et développer une application multicouche répartie en intégrant les recommandations de sécurité

La compétence « **Concevoir une application** » est l'objet même de l'architecture présentée en section 10 : une application répartie entre trois clients et une API centrale, et un backend organisé en couches aux dépendances strictement orientées. La compétence « **Développer des composants métier** » correspond aux services qui portent les règles du sujet : le service de stock et son déstockage FIFO par prix, le service de calcul des butées et des échéances, le service de génération des jetons — tous présentés avec leur code en section 17. La compétence « **Construire une application organisée en couches** » se vérifie dans la solution .NET elle-même, découpée en trois projets (Core, Infrastructure, API) dont les références se contrôlent à la compilation.

La compétence « **Développer une application mobile** » est couverte par l'application Android native en Kotlin décrite en section 14 : architecture MVVM, appels réseau en coroutines via Retrofit, stockage chiffré du jeton et lecture de QR codes. Enfin, la compétence « **Préparer et exécuter les plans de tests d'une application** » correspond à la démarche de validation de la section 19 : dix-huit scénarios de tests couvrant les règles de gestion, exécutés au niveau de l'API puis des interfaces, et un jeu d'essai documenté sur la fonctionnalité la plus représentative du projet, la consommation FIFO du stock.

3. Expression des besoins

Cette section reformule le cahier des charges fourni par Hesias afin de définir les objectifs et les limites du projet. Ce travail de reformulation n'est pas une simple paraphrase : c'est en réécrivant chaque exigence avec mes propres mots que j'ai identifié les questions de conception qu'elle soulevait, et c'est cette version qui a servi de base à la définition des fonctionnalités essentielles, à leur priorisation, puis aux spécifications de la section 4.

3.1. Objectifs fonctionnels

Comptes utilisateurs

L'application est utilisée par plusieurs personnes aux fonctions différentes — techniciens, responsable magasin, gestionnaire de flotte. Chaque utilisateur est identifié par un nom, un prénom, une fonction et un email, et dispose d'un couple login / mot de passe créé en base de données. Le cahier des charges n'exigeait pas de gestion fine des niveaux d'accès : tous les utilisateurs ont tous les droits, la gestion des rôles étant proposée en bonus. J'ai néanmoins mis en place une véritable authentification par jeton JWT avec mots de passe hachés, et ce pour une raison métier autant que technique : le sujet exige que toutes les opérations de stock soient horodatées et identifiées — savoir « qui a fait quoi, et quand » suppose de savoir de manière fiable qui est connecté.

Gestion du magasin (stock)

Le magasin suit les consommables disponibles pour l'entretien : filtres, plaquettes, huile, liquides. Chaque pièce est identifiée par une **catégorie**, une **marque** et une **référence** — par exemple : Filtre à huile – Purflux – LS923. À la saisie, l'application propose par autocomplétion les catégories, marques et références déjà connues, tout en permettant d'en saisir de nouvelles ; un bouton d'édition permet de corriger une dénomination erronée après coup. Chaque pièce possède un seuil minimum, à zéro par défaut ; lorsqu'il est différent de zéro et que le stock disponible passe en dessous, une alerte de stock est affichée à l'utilisateur.

L'utilisateur enregistre des **entrées de stock** en précisant la quantité et le prix de revient unitaire, et peut ajuster la quantité disponible pour refléter la réalité en cas de casse ou de perte. La consommation, elle, se fait généralement depuis les autres écrans, lors de la saisie des opérations d'entretien. Une règle métier singulière structure tout le module : **le stock consomme en priorité les lots les moins chers d'une pièce**. Si des filtres ont été saisis pour 10 unités à 4,20 € puis pour 20 unités à 5,60 €, ceux à 4,20 € sont déstockés d'abord. Cette règle, anodine en apparence, a des conséquences profondes sur la conception de la base de données, détaillées en section 11. Enfin, toutes les opérations effectuées sur le stock sont horodatées et identifiées.

Véhicules et opérations

Les véhicules sont identifiés par une immatriculation — ancien et nouveau format —, une marque et un modèle. Certains véhicules ne sont pas immatriculés, typiquement les engins de chantier : ils reçoivent alors un identifiant libre choisi par l'exploitant. Chaque véhicule a un kilométrage, que l'on suppose mis à jour par une solution externe, télémétrie embarquée ou saisie humaine régulière ; l'API expose pour cela un point d'entrée dédié. En bonus, des propriétés arbitraires peuvent être attachées à un véhicule — type moteur, type de boîte de vitesses — sans que la structure de la base n'ait à être modifiée.

Le cœur fonctionnel est le **journal** de chaque véhicule, qui recense les opérations effectuées. Une opération comporte une date, un kilométrage, un libellé, éventuellement des consommables du magasin dont la quantité est précisée et qui sont automatiquement retirés du stock, éventuellement des pièces détachées saisies librement avec leur prix de revient, et éventuellement une **butée** — calendrier ou kilométrique — indiquant quand l'opération devra être renouvelée. Des **méta-opérations** regroupent plusieurs opérations saisies ensemble : la « Vidange moteur » englobe par exemple le vidage de l'ancienne huile, le remplacement du joint de carter, celui du filtre à huile et l'ajout d'huile neuve. Une butée peut être posée sur la méta-opération plutôt que sur ses composantes. Des **notes** complètent le journal : libres, datées et kilométrées, ou rattachées à une opération ou une méta-opération.

Le sujet insiste enfin sur deux exigences d'usage. D'une part, l'application doit **assister la saisie** : un kilométrage saisi pour une opération devient la valeur par défaut des opérations suivantes, de même pour la date — le technicien qui enchaîne cinq opérations sur le même véhicule ne doit pas ressaisir cinq fois les mêmes informations. D'autre part, lorsqu'une opération arrive en butée, l'utilisateur doit en être **notifié** par le système de notification de l'application, et la situation des échéances doit être lisible de deux manières complémentaires : en incrément — « dans 6 mois, dans 6 000 km » — et en absolu — « le 15/03/2027, à 145 967 km ».

Application mobile

L'application mobile est un outil de saisie « live » : une fois connecté, le technicien choisit un véhicule — avec, en bonus, une reconnaissance automatique par plaque, code-barres ou tag NFC —, consulte son journal pour vérifier les notes laissées par ses collègues, puis saisit les opérations et méta-opérations effectuées au fil de son intervention. L'application doit réduire au maximum le besoin de saisie, en reprenant les mêmes mécanismes d'assistance que la version web.

3.2. Limites et périmètre

Conformément au cahier des charges, certains éléments sont volontairement hors périmètre ou simplifiés, et il importe de les nommer pour définir honnêtement les limites du projet. Il n'y a pas de gestion de rôles ni de niveaux d'accès : tous les utilisateurs authentifiés ont tous les droits, la gestion fine des permissions étant un bonus que je n'ai pas retenu et que l'on retrouve dans les axes d'amélioration. Le kilométrage des véhicules est réputé mis à jour par une solution externe : l'API expose l'endpoint nécessaire, mais aucune télémétrie réelle n'a été développée. L'édition de rapports PDF depuis la version web, proposée en bonus par le sujet, n'a pas été retenue dans cette version. Enfin, si le magasin peut techniquement référencer des pièces détachées, celles-ci sont généralement saisies « au vol » et à l'unité au niveau des opérations, sans lien avec le stock — c'est le fonctionnement voulu par le sujet pour l'entretien d'une flotte.

3.3. Contraintes

Le projet s'est construit sous cinq contraintes principales. La première est technique : l'application web doit s'exécuter dans un container Docker, ce qui a orienté très tôt la stratégie de build et de déploiement du front. La deuxième est méthodologique : une maquette de tous les écrans devait être réalisée avant le développement. La troisième tient au périmètre : trois supports à livrer — web, desktop, mobile — partageant le même socle de données, ce qui, pour un développeur seul, impose des choix d'architecture qui maximisent la réutilisation. La quatrième est documentaire : un plan de

tests écrit doit décrire les scénarios et les données de validation finale. La cinquième, enfin, est humaine : le projet est individuel, mené en parallèle des cours et de l'entreprise, sur la période de juillet à août 2026 — un calendrier serré qui a directement façonné la gestion des priorités décrite en section 5.

3.4. Livrables attendus

Le rendu final du Major Project comprend une archive du code source de chaque partie du projet — SQL et migrations, web, desktop, mobile — incluant les systèmes de build correspondants : fichiers de solution .NET, configuration npm, Angular et Electron, projets Gradle. S'y ajoutent le document de plans de tests au format PDF et le présent dossier de projet, rédigé selon le plan type du référentiel du titre. Le tout se conclut par une présentation orale devant un jury de professionnels, préparée à partir de ce dossier et ouverte par le résumé en anglais de la section 1.1.

4. Spécifications fonctionnelles

Cette section formalise les règles de gestion issues de l'expression des besoins, telles qu'elles sont réellement implémentées dans l'application. Chaque règle porte un identifiant — RG-U pour les utilisateurs, RG-M pour le magasin, RG-V pour les véhicules et le journal — afin de pouvoir être référencée sans ambiguïté par les scénarios du plan de tests de la section 19. Les règles sont ici rédigées et commentées, car leur intérêt tient moins à leur énoncé qu'aux décisions de conception qu'elles ont entraînées.

4.1. Règles de gestion — authentification et utilisateurs

Un utilisateur est défini par son nom, son prénom, sa fonction, son email, son login et son mot de passe (**RG-U1**) ; l'email et le login sont uniques, et cette unicité est garantie par la base de données elle-même au moyen d'index, et non par une simple vérification applicative qui pourrait être contournée. Le mot de passe n'est jamais stocké ni transmis en clair : seul son hash BCrypt est conservé (**RG-U2**), et ce hash ne sort jamais de l'API — aucun DTO de réponse ne le contient. Une connexion réussie délivre un jeton JWT valable deux heures, que toute requête ultérieure doit présenter (**RG-U3**). Un échec de connexion renvoie un message unique, identique que le login soit inconnu ou que le mot de passe soit erroné (**RG-U4**) : ce détail, recommandé par l'OWASP, empêche un attaquant d'énumérer les comptes existants. Enfin, conformément au périmètre du sujet, tous les utilisateurs authentifiés disposent des mêmes droits (**RG-U5**).

4.2. Règles de gestion — magasin

Une pièce est identifiée par le triplet catégorie, marque et référence (**RG-M1**). À la saisie d'une pièce, les valeurs déjà connues pour chacun de ces trois champs sont proposées par autocomplétion, tout en laissant la possibilité d'en saisir de nouvelles (**RG-M2**) ; une dénomination erronée reste corrigable à tout moment grâce au bouton d'édition de la pièce (**RG-M3**). Chaque pièce porte un seuil minimum, à zéro par défaut : dès que ce seuil est strictement positif et que le stock disponible passe en dessous, une alerte de stock est affichée sur le tableau de bord et dans le magasin (**RG-M4**).

Le stock lui-même obéit à une logique de lots. Chaque entrée de stock crée un lot portant sa quantité et son prix de revient unitaire (**RG-M5**). La consommation retire d'abord les lots les moins chers et, à prix égal, les plus anciens : c'est le déstockage FIFO par prix (**RG-M6**), la règle la plus caractéristique du sujet. Une consommation supérieure au stock disponible est purement et simplement refusée, sans qu'aucun lot ne soit modifié (**RG-M7**) — il n'existe pas de stock négatif dans Hesias Fleet. L'ajustement, troisième type de mouvement, fixe le stock à la quantité réelle constatée lors d'un inventaire, après une casse ou une perte (**RG-M8**). Enfin, règle transversale à tout le module : chaque mouvement, qu'il s'agisse d'une entrée, d'une consommation ou d'un ajustement, est horodaté et rattaché à l'utilisateur qui l'a effectué (**RG-M9**), ce qui rend l'historique du magasin intégralement auditable.

4.3. Règles de gestion — véhicules et journal

Un véhicule porte une immatriculation à l'ancien et/ou au nouveau format ou, à défaut, un identifiant libre choisi par l'exploitant (**RG-V1**) — les trois champs sont optionnels individuellement, mais un véhicule doit être identifiable par au moins l'un d'eux. Son kilométrage est mis à jour par un

point d'entrée dédié de l'API, prévu pour la solution externe de télémétrie ou de saisie (**RG-V2**). Des propriétés arbitraires sous forme de paires clé/valeur peuvent lui être attachées, comme le type moteur ou la boîte de vitesses (**RG-V3**).

Une opération du journal comporte une date, un kilométrage et un libellé, et en option des consommables du magasin, des pièces libres avec leur prix, et une butée (**RG-V4**). La saisie d'une opération comportant des consommables déclenche leur retrait du stock selon les règles RG-M6 et RG-M7 (**RG-V5**) : si le stock est insuffisant pour l'un des consommables, c'est toute la création de l'opération qui est refusée avec un message explicite. Une butée est calendaire, kilométrique, ou les deux à la fois (**RG-V6**). Une méta-opération regroupe plusieurs opérations et peut porter elle-même une butée (**RG-V7**). La suppression d'une opération détache ses liens sans suppression en cascade : une méta-opération survit à la disparition de l'une de ses composantes (**RG-V8**). Une note est libre — datée et éventuellement kilométrée — ou rattachée à une opération ou une méta-opération, et apparaît dans le journal dans tous les cas (**RG-V9**).

Les deux dernières règles portent sur le temps. Une opération est « due » dès qu'au moins une de ses butées est atteinte ou dépassée, et l'utilisateur en est alors notifié (**RG-V10**) ; les échéances sont systématiquement présentées sous deux formes, en incrément — dans tant de jours, dans tant de kilomètres — et en absolu — à telle date, à tel kilométrage (**RG-V11**). Enfin, l'assistance à la saisie : la date et le kilométrage de la dernière saisie deviennent les valeurs par défaut des saisies suivantes (**RG-V12**), sur le web comme sur le mobile.

4.4. Cas d'utilisation principaux

Deux acteurs structurent les cas d'utilisation : le **gestionnaire de flotte**, qui travaille sur la version web ou desktop, et le **technicien**, qui utilise principalement le mobile en atelier mais peut également accéder au web. Tous deux commencent par se connecter avec leur login et leur mot de passe, et peuvent se déconnecter à tout moment depuis n'importe quel support.

Le gestionnaire consulte le tableau de bord pour prendre connaissance des opérations dues et des alertes de stock. Il gère le référentiel des véhicules : création à l'arrivée d'un véhicule dans la flotte, édition de ses informations, consultation de sa fiche, ajout de propriétés techniques. Il tient le journal de chaque véhicule en y saisissant opérations, méta-opérations et notes, et consulte l'historique et la situation des butées. Il gère enfin le magasin : création et correction des pièces, enregistrement des entrées de stock avec leur prix de revient, ajustements d'inventaire, suivi des alertes.

Le technicien, lui, suit un parcours plus court et plus répétitif, optimisé pour l'atelier : il identifie un véhicule — en le cherchant dans la liste ou en scannant son QR code —, consulte son journal pour lire les notes laissées par ses collègues, puis saisit au fil de l'intervention les opérations effectuées, avec leurs consommables. Chaque saisie mobile est immédiatement visible sur le web : il n'y a qu'une seule source de données, l'API.

5. Gestion de projet : comment s'est déroulé le projet ?

5.1. Démarche générale

J'ai commencé par **définir les fonctionnalités essentielles** de l'application en me basant sur le sujet initial, ce qui m'a permis de structurer clairement le travail à réaliser : gestion du magasin avec déstockage FIFO, journal des véhicules, butées et notifications, saisie assistée, déclinaison mobile. Ce travail de décomposition, mené sur le papier avant tout choix technique, a produit la liste des règles de gestion présentée en section 4 — et c'est elle qui a servi de colonne vertébrale à tout le reste.

Ensuite, j'ai **établi des priorités** pour les différentes tâches afin de minimiser les oublis, en incluant non seulement le développement, mais aussi la création d'une identité visuelle et les maquettes de l'application. La règle que je me suis fixée tient en une phrase : le socle de données et l'API d'abord, car les trois clients en dépendent ; les interfaces ensuite ; les finitions — identité graphique, application desktop — en dernier. Cette règle peut sembler évidente, mais elle demande de la discipline : il est tentant de commencer par les écrans, qui donnent un résultat visible immédiatement, alors que c'est la stabilité du contrat d'API qui conditionne la vitesse de tout le développement ultérieur.

J'ai également **investi du temps pour apprendre et maîtriser** les langages et frameworks nécessaires — C# et ASP.NET Core avec Entity Framework Core pour le backend, Angular et TypeScript pour le frontend, Kotlin pour le mobile — afin de garantir une base technique solide pour le projet. Concrètement, chaque nouvelle brique a été précédée d'une demi-journée de lecture de la documentation officielle et de prototypage jetable, hors du dépôt : un mini-projet Angular pour comprendre les signals et les composants standalone, un écran Kotlin isolé pour prendre en main Retrofit et les coroutines. Ce temps « perdu » en apparence a été largement récupéré, car très peu de code du projet a dû être réécrit.

Le développement s'est enfin déroulé en **phases successives** et clairement bornées : d'abord la création de l'API complète avec ASP.NET Core — métier, stock FIFO, butées, authentification JWT —, puis la conception de l'interface utilisateur web avec Angular, ensuite l'application mobile Android, et enfin l'application desktop Electron accompagnée du restylage complet de l'interface. Chaque phase se terminait par un état stable, testé et commité, avant que la suivante ne commence.

5.2. Planning et suivi

Le suivi du projet s'est appuyé sur Git et GitHub : chaque étape correspond à des commits explicites qui permettent de retracer l'avancement réel, et c'est cet historique qui me sert ici de source pour reconstituer le déroulement — un journal de bord que je n'ai pas eu à tenir séparément, puisque le dépôt le tenait pour moi.

Début juillet 2026 — cadrage et conception. Les deux premières semaines ont été consacrées à la lecture approfondie du sujet, à la définition des fonctionnalités essentielles et à leur priorisation, à la création de l'identité visuelle et des maquettes de tous les écrans, et à la conception du modèle de données. Aucune ligne de code applicatif n'a été écrite pendant cette période, et c'est un choix assumé : les erreurs de conception coûtent d'autant moins cher qu'elles sont détectées tôt, et c'est précisément pendant cette phase que la modélisation du stock en lots — indispensable au FIFO par prix — a été trouvée et validée sur des exemples chiffrés.

Mi-juillet 2026 — la base de données. Mise en place de PostgreSQL 16 dans Docker via docker-compose, écriture des entités C# du domaine, et génération de la migration initiale le 15 juillet. À partir de cette date, le schéma complet des onze tables existait et n'a plus subi de modification structurelle majeure — signe que la phase de conception avait joué son rôle.

23 juillet 2026 — l'API complète. Le commit de cette date, explicitement intitulé « API complète : métier + stock FIFO + butées + auth JWT », marque la fin de la phase backend : l'ensemble des controllers, les DTOs, les repositories, les services métier de stock et de butées, l'authentification par jeton et la documentation Swagger étaient en place et testés. C'est le jalon le plus important du projet : à partir de là, le contrat REST était stable, et les trois clients allaient pouvoir se développer contre lui sans retour en arrière.

30 juillet 2026 — le frontend Angular. Une semaine plus tard, la SPA était fonctionnelle : écran de connexion, tableau de bord, gestion des véhicules et magasin. Cette phase a occasionné les seuls allers-retours notables sur l'API : l'ajout d'endpoints d'édition en PUT pour les véhicules et les pièces, dont le besoin est apparu en construisant les écrans — il est difficile d'anticiper depuis le backend tout ce qu'une interface exigera.

4 août 2026 — l'application mobile. Développement de l'application Android en Kotlin : connexion, liste des véhicules, journal et saisie d'opérations. Cette phase s'est conclue par mon premier vrai conflit de merge Git, raconté en 5.4.

17 août 2026 — le desktop et l'identité. Ajout de l'application desktop Electron réutilisant le build Angular, et restylage complet du front aux couleurs de l'identité Hesias Fleet. Placer ce restylage en fin de projet était délibéré : il aurait été absurde de peaufiner deux fois les mêmes écrans.

23 août 2026 — les finitions. Derniers compléments fonctionnels — autocomplétion du magasin, édition des pièces, mise à jour du kilométrage côté API —, corrections sur le mobile, et rédaction du présent dossier.

Avec le recul, ce découpage a bien fonctionné : au moment d'attaquer le frontend, l'API était déjà stable et documentée par Swagger, ce qui m'a permis de développer les trois clients contre le même contrat REST sans retouche majeure du backend. La leçon que j'en retire est générale : dans un projet multi-clients, la stabilité du contrat d'échange vaut plus que l'avancement apparent de n'importe quelle interface.

5.3. Objectifs de qualité

Cinq objectifs de qualité ont guidé le développement au quotidien. Le premier est la **lisibilité et la maintenabilité** du code : architecture en couches, nommage explicite, et commentaires en français qui expliquent le « pourquoi » des règles métier directement là où elles sont implémentées — un choix visible dans tous les extraits de la section 17. Le deuxième est la **cohérence du contrat REST** : des DTOs dédiés en entrée et en sortie, des conventions d'URL uniformes et des codes HTTP appropriés, pour que les trois clients puissent traiter l'API de manière prévisible. Le troisième est la **sécurité dès la conception** plutôt qu'ajoutée après coup : mots de passe hachés, jetons signés, stockage chiffré sur mobile, messages d'erreur non révélateurs. Le quatrième est la **traçabilité** : tout mouvement de stock est horodaté et rattaché à un utilisateur, sans exception. Le cinquième, enfin, est la **validation par scénarios** : un plan de tests manuel exécuté via Swagger et les interfaces, avec un jeu d'essai documenté, présenté en section 19.

5.4. Difficultés rencontrées et arbitrages

Mener trois clients de front avec un seul développeur. Le risque principal du sujet était de disperser l'effort entre web, desktop et mobile, et de finir avec trois applications inachevées plutôt qu'une solution complète. L'arbitrage structurant a été de tout faire reposer sur une API unique développée en premier, puis de choisir Electron pour le desktop afin de réutiliser à cent pour cent le front Angular : le « troisième support » n'a finalement coûté qu'une journée de travail — fenêtre, packaging, durcissement de sécurité — au lieu d'une application complète. Sans cet arbitrage, le calendrier n'aurait pas tenu.

La règle FIFO par prix. Ma première modélisation du stock — une simple quantité par pièce — ne permettait ni d'appliquer la règle « consommer les moins chers d'abord », ni de valoriser les consommations au prix réel des lots concernés. Il a fallu revenir sur la conception et introduire la notion de lot avant d'écrire le service de déstockage. Cette itération, faite au moment de la migration initiale et validée sur des exemples chiffrés, a évité une refonte bien plus coûteuse en cours de projet ; elle m'a aussi appris qu'une règle métier d'apparence anecdotique peut dicter la structure même de la base.

Les fuseaux horaires. PostgreSQL et .NET manipulent des timestamps UTC alors que l'interface affiche des dates locales — un terrain classique de bugs sournois, où une opération saisie le 1er du mois s'affiche la veille au soir. La règle adoptée est simple et sans exception : tout est stocké et échangé en UTC, avec le suffixe Z dans les JSON, et la conversion n'a lieu qu'à l'affichage — pipes de date côté Angular, utilitaires dédiés côté mobile. Le calcul des butées en jours se fait quant à lui sur des dates tronquées à minuit, pour éviter les décalages d'un jour selon l'heure de la journée.

Un merge Git conflictuel. Le développement du mobile en parallèle de retouches du README a produit mon premier vrai conflit de merge, visible dans l'historique du 4 août. Plutôt que d'écraser une branche pour aller vite, j'ai pris le temps de le résoudre proprement — lecture des marqueurs de conflit, choix ligne à ligne, commit de conciliation explicite. C'est un petit événement à l'échelle d'un projet, mais une compétence réelle en entreprise, où les conflits de merge sont quotidiens.

5.5. Méthode de travail et rythme

Projet individuel oblige, je n'ai pas déroulé une méthode agile complète — il n'y a ni daily meeting ni rétrospective quand on est seul —, mais j'en ai gardé les outils utiles à une personne : un **backlog priorisé**, tenu dès la phase de cadrage, où chaque fonctionnalité du sujet est devenue une tâche marquée « socle », « essentiel » ou « bonus » ; des **jalons hebdomadaires** correspondant aux phases du planning ; et une **définition du fini** constante — une fonctionnalité n'est terminée que lorsqu'elle est testée via Swagger ou l'interface, commitée avec un message explicite, et rayée du backlog.

Chaque session de travail commençait par la relecture du backlog et se terminait par un commit, si bien que l'historique Git fait office de journal de bord fidèle. Cette discipline m'a évité les deux pièges classiques du projet solo : l'oubli de fonctionnalités — tout est dans le backlog, y compris l'identité visuelle et les maquettes — et le travail en vrac — chaque commit correspond à un lot cohérent et nommé. Le rythme a alterné des semaines de développement intensif, comme celle de l'API en juillet, et des séquences plus courtes consacrées à l'apprentissage, selon le principe du prototypage jetable décrit en 5.1.

5.6. Gestion des risques

Six risques principaux avaient été identifiés en début de projet, et chacun a reçu une parade explicite. Le premier — **sous-estimer la charge des trois supports** — pouvait conduire à un projet inachevé ; il a été traité par l'architecture elle-même : API unique développée d'abord, desktop obtenu par réutilisation du front via Electron, mobile limité au parcours technicien décrit par le sujet. Le deuxième — **découvrir une technologie en cours de développement** — menaçait la qualité du code autant que le calendrier ; la parade a été le temps d'apprentissage planifié avant chaque brique et les prototypes jetables réalisés hors du dépôt.

Le troisième risque — **une mauvaise conception du stock** — était le plus dangereux car il aurait imposé une refonte de la base en cours de projet ; il a été neutralisé en validant la modélisation par lots sur des exemples chiffrés avant la migration initiale. Le quatrième — **la perte de code ou de données** — est couvert par les commits et push réguliers sur GitHub, par le volume Docker nommé qui préserve la base entre les redémarrages, et par la migration rejouable qui permet de reconstruire une base vierge à tout moment. Le cinquième — **la dérive du périmètre**, tentation permanente face aux bonus attirants du sujet — a été contenu par le backlog en trois niveaux : les bonus, comme le scan QR ou les propriétés libres, n'ont été traités qu'une fois l'essentiel livré. Le sixième, enfin — **les incohérences entre clients** — a été prévenu par le contrat REST unique documenté dans Swagger, par des modèles TypeScript et Kotlin alignés sur les DTOs, et par des tests croisés vérifiant qu'une saisie mobile apparaît bien sur le web.

5.7. Estimation et répartition de la charge

En fin de projet, il est instructif de comparer la répartition réelle du temps à l'intuition de départ. La phase de cadrage et de conception — sujet, priorisation, maquettes, modèle de données — a occupé environ un cinquième du temps total, ce qui peut sembler beaucoup pour une période sans production de code, mais qui s'est révélé être le meilleur investissement du projet : aucune refonte structurelle n'a été nécessaire ensuite. Le backend — base, API, services métier, authentification — a représenté le bloc le plus important, environ un tiers du temps, dont une part significative sur le seul module de stock, sa règle FIFO et sa traçabilité. Le frontend web a pris un peu moins d'un quart, largement grâce à Angular Material qui a évité de développer des composants d'interface de base. Le mobile a occupé environ un sixième du temps, l'API étant déjà stable et le périmètre volontairement resserré sur le parcours technicien. Le desktop, enfin, a été la meilleure affaire du projet : à peine quelques pourcents du temps total pour un support complet, grâce à la réutilisation du front par Electron.

Deux écarts entre estimation et réalité méritent d'être notés, car ils sont instructifs. J'avais sous-estimé le temps de mise au point des détails d'ergonomie — valeurs par défaut, autocomplétion, rafraîchissements après fermeture des popups — qui paraissent mineurs sur une maquette mais consomment de vraies journées en développement. J'avais en revanche surestimé la difficulté de l'authentification JWT : l'écosystème ASP.NET Core la rend très abordable dès lors qu'on lit la documentation dans l'ordre. La leçon générale rejoint celle de la section 5.2 : ce sont rarement les gros morceaux annoncés qui débordent, mais l'accumulation des finitions — d'où l'intérêt d'un backlog qui les liste explicitement au lieu de les laisser à l'état de « détails ».

6. Maquettage et identité visuelle

6.1. La démarche de maquettage

Conformément au sujet, tous les écrans ont été maquetés avant le développement. Les maquettes ont couvert les trois supports : pour le web et le desktop, l'écran de connexion, le tableau de bord, la liste et la fiche des véhicules avec ses trois onglets, le magasin et l'ensemble des fenêtres de saisie ; pour le mobile, les quatre écrans du parcours technicien — connexion, liste des véhicules, journal, saisie d'opération.

Le maquettage a joué pleinement son rôle de révélateur, et trois décisions majeures de l'interface finale en sont directement issues. C'est en dessinant les écrans qu'est apparu le besoin d'un tableau de bord dédié aux alertes : ma première esquisse dispersait les notifications dans les écrans concernés, et il devenait évident sur le papier qu'un utilisateur pressé ne les verrait jamais. C'est également le maquettage qui a fait émerger le découpage de la fiche véhicule en onglets — la version « une seule page » devenait illisible dès qu'un véhicule avait un peu d'historique. Enfin, c'est en simulant sur papier la saisie de cinq opérations à la suite qu'est venu le choix des fenêtres de dialogue plutôt que des pages dédiées, afin de préserver le contexte de l'utilisateur entre deux saisies.

La structure de navigation issue des maquettes — la barre latérale à trois entrées — est celle de l'application finale, sans modification. Les maquettes ont ainsi servi de contrat visuel pour tout le développement du front : à chaque écran développé, la question n'était pas « que faut-il faire ? » mais « la réalisation est-elle conforme à ce qui a été dessiné ? », ce qui est une position beaucoup plus confortable et rapide.

6.2. L'identité visuelle

Hesias Fleet dispose d'une identité visuelle propre, appliquée lors du restylage complet du front réalisé le 17 août, une fois les fonctionnalités stabilisées. La palette s'articule autour d'une dominante **vert « flotte »** pour les actions principales et les éléments actifs — un vert profond qui évoque à la fois l'univers de l'entretien mécanique et une forme de fiabilité —, d'un **aplat charbon** pour la barre latérale, qui ancre visuellement la navigation, et de cartes claires sur fond neutre pour le contenu. La typographie et les composants Material assurent la cohérence d'ensemble et les standards d'accessibilité.

Les états d'alerte — opérations dues, stock sous le seuil — utilisent systématiquement la couleur d'erreur Material, un rouge appliqué en liseré et sur les chiffres concernés : l'œil est attiré là où il le faut, sans que l'interface ne crie en permanence. La même identité est déclinée sur les trois supports : le desktop Electron réutilise le front web tel quel, et l'application Android reprend la palette au travers des Material Components. Ce travail d'identité faisait partie des tâches planifiées dès le départ ; le placer en fin de projet, une fois les écrans fonctionnellement figés, a évité de payer deux fois le même travail de mise en forme.

6.3. Du papier à l'écran : ce que les maquettes n'avaient pas prévu

Si la structure issue des maquettes a tenu jusqu'au bout, l'honnêteté oblige à dire que tout n'était pas anticipé. Trois ajustements sont apparus en cours de développement. Le premier concerne les

tableaux : sur maquette, les colonnes du journal semblaient suffisantes ; à l'usage, avec de vraies données, il a fallu hiérarchiser — mettre le libellé en avant, reléguer les détails dans les panneaux dépliables — pour que l'œil trouve l'information. Le deuxième concerne les états vides : les maquettes montrent toujours des écrans remplis, mais la première impression d'un utilisateur réel est un tableau de bord sans données ; il a fallu dessiner après coup des états vides accueillants, qui indiquent quoi faire plutôt que d'afficher des tableaux déserts. Le troisième concerne les messages d'erreur, absents des maquettes par nature, et pourtant décisifs pour la confiance : chaque refus de l'API — stock insuffisant, identifiants incorrects — a reçu une formulation explicite côté interface.

J'en retire une conviction pour les prochains projets : maquetter aussi les états limites — écran vide, erreur, chargement — et pas seulement le cas nominal. C'est un petit effort au moment du maquetage, et cela évite de concevoir ces états dans l'urgence, au milieu du développement.

7. Environnement de travail

Le projet a été développé seul, sur un poste Windows, avec un environnement outillé volontairement proche de celui d'une équipe professionnelle. Cette section décrit chaque outil et le rôle précis qu'il a joué.

GitHub et Git constituent le socle du projet : le dépôt Mric63/HesiasFleet héberge l'ensemble de la solution — backend, front, desktop et mobile dans un même monorepo —, en versionne chaque étape et permet la synchronisation entre machines. Au-delà de la sauvegarde, Git a structuré la méthode de travail elle-même, comme décrit en section 5 : les commits nommés par phase tiennent lieu de journal de bord, et l'historique du dépôt est la source des dates du planning présenté plus haut.

JetBrains Rider a été l'IDE principal pour tout le backend C# / .NET. Son intérêt dans ce projet tient à trois choses : la navigation dans une solution multi-projets — passer d'une interface du domaine à son implémentation dans Infrastructure en un raccourci —, le débogueur intégré qui permet de poser des points d'arrêt dans un service pendant que le front appelle l'API, et l'intégration des outils Entity Framework Core pour générer et appliquer les migrations. J'y ai également conservé une collection de requêtes de test au format .http, rejouable à tout moment contre l'API.

JetBrains WebStorm a servi au frontend Angular / TypeScript ainsi qu'à la partie Electron : refactorings TypeScript fiables, intégration de l'Angular CLI, analyse statique permanente. **Android Studio** — lui-même construit sur la plateforme IntelliJ — a accueilli l'application mobile Kotlin : éditeur de layouts XML, gestion Gradle, émulateur Android depuis lequel l'API de la machine hôte est atteinte via l'adresse spéciale 10.0.2.2, et profileur réseau pour observer les échanges Retrofit. Le choix de cette suite cohérente d'IDE JetBrains n'est pas anecdotique : mêmes raccourcis, même ergonomie de recherche et de refactoring quel que soit le langage, ce qui réduit sensiblement le coût mental de passer plusieurs fois par jour du C# au TypeScript puis au Kotlin.

Docker Desktop a rempli deux fonctions distinctes : exécuter PostgreSQL 16 en container au quotidien — la base du projet n'a jamais été installée directement sur le poste — et construire l'image de production du front web, selon le Dockerfile multi-stage décrit en section 20. **Google Chrome** a été l'environnement de test de l'application web, et surtout ses DevTools : l'onglet Réseau pour inspecter les requêtes et vérifier la présence du Bearer token, la console pour le débogage, et le mode responsive pour contrôler la tenue des écrans à différentes tailles.

Swagger UI, généré automatiquement par l'API, mérite une mention particulière : il a été utilisé quotidiennement pendant toute la phase backend, avant même l'existence du moindre écran. Chaque endpoint y est décrit avec ses DTOs, et le bouton « Authorize » permet d'y coller un jeton JWT pour tester les endpoints protégés — c'est l'outil qui a permis de valider l'API de manière autonome, puis de ressaisir le jeu de données de test après chaque réinitialisation de la base. Enfin, **pgAdmin et psql** ont permis le contrôle direct du contenu de PostgreSQL : vérification du schéma créé par les migrations, inspection des lots et des mouvements de stock pendant la mise au point du FIFO, et validation des données du jeu d'essai.

8. Environnement technique

Cette section détaille précisément les briques techniques du projet et leurs versions. Un point de méthode d'abord : toutes les versions citées ici sont les versions **effectivement résolues** dans les fichiers du dépôt — fichiers de projet .csproj, fichier de verrouillage package-lock.json côté web, scripts Gradle côté mobile, Dockerfile et docker-compose — et non des plages de compatibilité approximatives. C'est une exigence que je me suis fixée pour ce dossier : chaque version doit pouvoir être vérifiée dans le code source livré.

8.1. Le backend et sa base de données

L'API est écrite en **C# sur .NET 8.0**, la version LTS de la plateforme, dont le support long terme garantit la stabilité du socle. Le framework web est **ASP.NET Core 8** — controllers, middleware, injection de dépendances intégrée. L'accès aux données passe par **Entity Framework Core 8.0.29** (packages Design et Tools pour les migrations) et par le provider **Npgsql.EntityFrameworkCore.PostgreSQL 8.0.11**, qui fait le pont vers la base. Trois bibliothèques complètent le backend : **BCrypt.Net-Next 4.2.0** pour le hachage des mots de passe, les packages **Microsoft.AspNetCore.Authentication.JwtBearer** et **System.IdentityModel.Tokens.Jwt** en version 8 pour la validation et la génération des jetons, et **Swashbuckle.AspNetCore 6.6.2** pour la documentation Swagger.

La base de données est **PostgreSQL 16**, exécutée dans un container Docker à partir de l'image officielle postgres:16, décrite dans le docker-compose du dépôt avec son volume nommé hesias_pgdata — les données survivent ainsi aux redémarrages du container — et exposée sur le port standard 5432. Le nullable reference typing de C# est activé sur les trois projets, ce qui fait vérifier par le compilateur la cohérence entre les colonnes optionnelles de la base et les propriétés nullables des entités.

8.2. Le frontend web et le desktop

Le frontend est développé avec **Angular 19.2.25** et son outillage **Angular CLI 19.2.27**, en **TypeScript 5.6.3**. La bibliothèque de composants est **Angular Material 19.2.19** avec le CDK associé, au design Material 3. La réactivité repose sur **RxJS 7.8.2** pour les flux HTTP et sur **zone.js 0.15.1** pour la détection de changements. Le build de production s'exécute sous **Node.js 22** — c'est l'image Docker node:22-alpine qui le réalise — et les fichiers produits sont servis par **nginx** (image alpine officielle), qui assure aussi le rôle de proxy vers l'API décrit en section 20.

L'application desktop repose sur **Electron 43.3.0**, empaquetée par **electron-builder 26.15.3** avec une cible Windows au format NSIS — un installateur classique. Elle n'introduit aucune dépendance d'interface supplémentaire, puisqu'elle charge le build Angular tel quel.

8.3. Le mobile Android

L'application mobile est écrite en **Kotlin 2.0.0**, construite par **Gradle 8.9** avec l'**Android Gradle Plugin 8.5.2** et compilée sous **JDK 17**. Elle cible le **SDK Android 34** (compileSdk et targetSdk) avec un minimum à l'API 24, ce qui la rend compatible avec les appareils depuis Android 7.0 — un choix pensé pour les téléphones d'atelier, rarement récents. Le réseau repose sur **Retrofit 2.11.0** avec le convertisseur Gson et l'intercepteur de journalisation **OkHttp 4.12.0** en debug ; l'asynchronisme sur

les **coroutines kotlinx 1.8.1** ; l'architecture sur les composants **AndroidX Lifecycle 2.8.4** (ViewModels). La sécurité du stockage utilise **AndroidX security-crypto 1.1.0-alpha06** et ses EncryptedSharedPreferences ; le scan de QR codes s'appuie sur **ZXing Android Embedded 4.3.0** ; l'interface sur **Material Components 1.12.0**.

9. Langages et technologies : choix et avantages

Le sujet laissant le choix des technologies, chaque brique a été retenue pour des raisons précises. Cette section les développe une à une, puis confronte chaque choix aux alternatives qui avaient été étudiées.

9.1. C# et ASP.NET Core 8 pour le backend

Le premier avantage d'ASP.NET Core est sa **maturité et ses performances** : c'est l'un des frameworks web les plus rapides du marché, et .NET 8 est une version LTS — un socle stable pour un projet évalué, sans risque de changement cassant en cours de route. Le deuxième est le **typage fort de C#** allié à un outillage de premier plan : une grande partie des erreurs est détectée à la compilation plutôt qu'à l'exécution, et l'activation des nullable reference types sur tout le projet fait vérifier par le compilateur la présence potentielle de valeurs nulles — précieux quand le modèle comporte autant de champs optionnels que les butées ou les immatriculations. Le troisième avantage est l'**injection de dépendances native** : enregistrer chaque repository et chaque service métier derrière son interface est l'affaire d'une ligne, et c'est ce mécanisme qui rend l'architecture en couches à la fois naturelle et testable. Le dernier est l'**écosystème** : authentification JWT, documentation Swagger, ORM — tout existe en packages officiels maintenus, sans dépendre de bibliothèques communautaires fragiles.

9.2. Entity Framework Core et PostgreSQL 16 pour les données

L'approche **code-first** d'Entity Framework Core est le premier argument : le schéma est décrit par les entités C# et versionné par des migrations, si bien que la base se crée et évolue de façon reproductible sur n'importe quel poste — un simple clone du dépôt suffit à reconstruire l'environnement de données. Le deuxième argument est la **sécurité par construction** : les requêtes LINQ sont systématiquement traduites en SQL paramétré, ce qui élimine mécaniquement les injections SQL classiques sans discipline particulière du développeur. Quant à **PostgreSQL**, c'est un SGBD open source robuste et complet — types riches, index, contraintes — dont l'image Docker officielle est éprouvée et dont le provider Npgsql pour EF Core est de première qualité ; il ne coûte rien, s'installe en une ligne de docker-compose, et n'impose aucun compromis fonctionnel au projet.

9.3. Angular 19 et Angular Material pour le web

Angular est un framework **structurant** : il impose une organisation — composants, services, routing, injection de dépendances — qui convient particulièrement bien à une application de gestion multi-écrans, et TypeScript apporte côté client le même confort de typage que C# côté serveur, jusqu'à faire des modèles du front un miroir vérifié des DTOs de l'API. Les apports propres à la **version 19** ont été réellement utilisés, et pas seulement subis : les composants standalone chargés paresseusement route par route allègent le bundle initial, et les signals simplifient considérablement la gestion d'état partagé — le compteur de notifications, par exemple, est un signal unique lu à la fois par la barre supérieure et par le tableau de bord. Enfin, **Angular Material** fournit d'emblée, avec un design cohérent et accessible, précisément les composants que le sujet réclame : tables de données, fenêtres de dialogue, onglets, champs à autocomplétion, badges de notification. Pour un développeur seul sur un calendrier court, c'est un gain de temps décisif sans sacrifice de qualité visuelle.

9.4. Electron 43 pour le desktop

Le choix d'Electron découle directement de la contrainte du projet : livrer une application desktop complète sans doubler le travail d'interface. Electron charge le **même build Angular** que le web dans une fenêtre native — une seule base de code d'interface à maintenir pour deux supports, et tout correctif appliqué au web immédiatement disponible sur desktop. La **distribution** est simple : electron-builder produit un installateur Windows NSIS en une commande. Quant aux critiques classiques d'Electron — poids de l'exécutable, consommation mémoire —, elles sont réelles mais peu pertinentes pour un outil de gestion interne installé sur quelques postes ; et les précautions de **sécurité** recommandées sont appliquées, comme détaillé en sections 13 et 18 : isolation de contexte, intégration Node désactivée, liens externes ouverts hors de l'application.

9.5. Kotlin et Android natif pour le mobile

Kotlin est le langage **officiellement recommandé** par Google pour Android, et ses qualités propres — null-safety, data classes, coroutines — rendent le code réseau à la fois concis et sûr : un appel d'API est une fonction suspendue de quelques lignes, sans callbacks imbriqués. Le développement **natif** donne un accès direct au matériel dont le projet a besoin : la caméra pour le scan de QR codes via ZXing, et le Keystore matériel du téléphone pour le chiffrement du jeton via security-crypto — sans couche d'abstraction supplémentaire à apprendre et à déboguer. Enfin, l'architecture **MVVM** — ViewModels, coroutines, Retrofit — est le standard industriel pour consommer une API REST depuis Android : la pratiquer telle quelle correspond exactement à la compétence mobile visée par le titre.

9.6. Docker pour la conteneurisation

Docker répond d'abord à une **exigence explicite du sujet** : l'application web doit s'exécuter dans un container. Le front est donc empaqueté par un Dockerfile multi-stage — build Angular sous Node 22, service des fichiers par nginx — décrit en section 20. Mais Docker apporte surtout la **reproductibilité** : la base PostgreSQL est décrite dans docker-compose avec son volume persistant, et l'environnement de données complet se recrée à l'identique en une commande, ce qui a servi tout au long du projet pour rejouer le jeu d'essai sur une base vierge. Enfin, le **proxy nginx** embarqué dans l'image de production sert les fichiers statiques Angular et relaie les appels /api vers l'API : vu du navigateur, tout provient de la même origine, et les problèmes de CORS disparaissent entièrement en production.

9.7. Les alternatives étudiées

Face à ASP.NET Core, l'alternative sérieuse était **Node.js avec Express** en TypeScript — cohérente avec le front, et que je connaissais déjà. Je lui ai préféré C# pour la solidité du typage côté serveur, l'injection de dépendances et l'authentification intégrées, et les migrations d'EF Core ; et aussi, assumons-le, pour progresser en C# avec Rider dans un contexte réel. Node n'a pas disparu du projet pour autant : c'est lui qui exécute le build Angular et l'outillage Electron.

Face à Angular, j'ai étudié **React avec Vite**. React est plus léger et plus répandu, mais il aurait fallu assembler soi-même routing, gestion de formulaires, appels HTTP et bibliothèque de composants — autant de choix et d'intégrations à assumer seul. Angular fournit ce cadre complet d'emblée, et Angular Material couvrait exactement les composants dont le sujet avait besoin ; à calendrier égal, le cadre intégré l'a emporté sur la flexibilité.

Pour la base de données, les alternatives étaient **MySQL** et **SQL Server**. SQL Server s'intègre naturellement à l'écosystème .NET, mais sa conteneurisation est plus lourde et sa licence moins simple pour un projet étudiant ; MySQL, de son côté, n'offrait pas d'avantage technique sur PostgreSQL, dont le provider EF Core est excellent. Pour le desktop, l'alternative native — **WPF ou MAUI** — aurait constitué un quatrième développement d'interface complet, incompatible avec le calendrier ; Electron réutilise tout. Pour le mobile, enfin, **Flutter et React Native** perdent leur argument principal — le cross-platform — dès lors que le sujet ne demande qu'une seule plateforme ; le natif Kotlin donnait un accès direct à la caméra et au Keystore, et correspondait à la compétence mobile du référentiel. Restait le choix du mécanisme d'authentification : face aux **sessions serveur avec cookies**, le jeton JWT porté par en-tête s'est imposé parce qu'il fonctionne à l'identique sur les trois clients — y compris le mobile natif, où la gestion de cookies serait artificielle — et qu'il garde l'API sans état.

10. Architecture de l'application

La solution est organisée autour d'une API REST centrale : les trois clients — web, desktop, mobile — consomment exactement le même contrat HTTP/JSON, et seule l'API accède à la base de données. Ce choix garantit que les règles métier — déstockage FIFO, calcul des butées, traçabilité — sont écrites et appliquées à un seul endroit : aucun client ne peut les contourner, et les corriger ne demande qu'une seule modification.

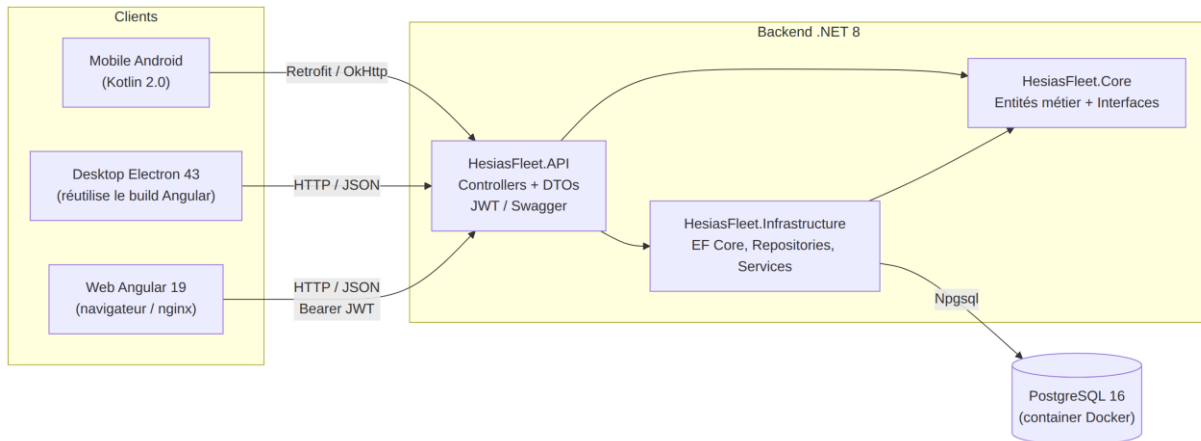


Figure 1 — Architecture générale de la solution

10.1. Un backend en trois couches

La solution .NET est découpée en trois projets aux dépendances strictement orientées, dans l'esprit de la « clean architecture ». Le projet **HesiasFleet.Core** est le domaine : il contient les entités métier — Vehicle, Operation, Part, StockEntry et les autres — ainsi que les interfaces des repositories et des services, comme `IStockService` ou `IDeadlineService`. Ce projet ne dépend de rien : ni d'Entity Framework, ni d'ASP.NET, ni d'aucune bibliothèque technique. Le projet **HesiasFleet.Infrastructure** est l'implémentation technique : le DbContext EF Core, les migrations, les repositories concrets et les services métier — `StockService`, `DeadlineService`, `TokenService`. Il ne dépend que de Core. Le projet **HesiasFleet.API**, enfin, est l'exposition HTTP : les controllers, les DTOs et toute la configuration — injection de dépendances, JWT, CORS, Swagger. Il dépend de Core et d'Infrastructure.

Les avantages de ce découpage sont concrets et se sont vérifiés pendant le projet. Les règles métier sont isolées et remplaçables : changer de SGBD ne toucherait que le projet Infrastructure, et le domaine n'en saurait rien. Les controllers restent minces : ils orchestrent les services, mais ne décident de rien — la section 17 le montre sur l'exemple de la création d'opération. Chaque service est enregistré dans le conteneur d'injection de dépendances au démarrage de l'application, derrière son interface : neuf enregistrements dans le fichier `Program.cs` relie les interfaces du domaine à leurs implémentations d'Infrastructure, et c'est ce mécanisme qui permettra, le jour venu, d'écrire des tests unitaires en substituant aux repositories de vraies doublures de test. Enfin, la direction des dépendances est vérifiée par le compilateur lui-même : le projet Core ne référence pas les autres, il est donc physiquement impossible d'y glisser une dépendance technique par inadvertance.

10.2. Un front décliné en deux supports

Le frontend Angular est découpé en **features** — authentification, tableau de bord, véhicules, magasin, opérations — et en un noyau **core** qui regroupe les services HTTP, les modèles, l'intercepteur et le guard de route. L'application desktop Electron n'est pas une seconde interface : elle charge le build Angular dans une fenêtre native, avec les mêmes services et le même comportement. Un correctif appliqué au web est donc immédiatement disponible sur desktop, et il n'existe qu'un seul endroit où l'interface peut se tromper.

10.3. Un mobile natif consommant la même API

L'application Android suit le patron **MVVM** : les Activities observent des ViewModels, qui appellent l'API au travers d'une interface Retrofit déclarant un endpoint par méthode. Un intercepteur OkHttp ajoute automatiquement le Bearer token à chaque requête — le symétrique exact de l'intercepteur HTTP Angular côté web. Cette symétrie délibérée entre les clients est développée en section 14.

10.4. Cheminement d'une requête, de bout en bout

Pour illustrer la collaboration des couches, suivons le trajet complet d'une action utilisateur — « enregistrer une opération avec un consommable » — depuis le clic jusqu'à la base de données. Tout commence dans la popup de saisie du front Angular : le formulaire réactif valide les champs, puis le service des opérations envoie le DTO en JSON sur le point d'entrée de création. Avant que la requête ne parte, l'intercepteur y ajoute l'en-tête d'autorisation portant le jeton ; en production, c'est nginx qui la reçoit et la relaie vers le container de l'API.

Côté serveur, le middleware JWT intervient en premier : il vérifie la signature du jeton, son émetteur, son audience et sa date d'expiration, puis établit l'identité de la requête. Le controller des opérations reçoit alors le DTO désérialisé et joue son rôle d'orchestrateur : pour chaque consommable demandé, il appelle le service de stock, qui applique la règle FIFO — lots les moins chers d'abord — et enregistre les mouvements correspondants ; si le stock est insuffisant, la création s'arrête là et le client reçoit un message exploitable. Sinon, le controller construit l'entité complète avec ses consommables et ses pièces libres, et la confie au repository. Entity Framework Core traduit l'ensemble en SQL paramétré et l'exécute sur PostgreSQL, dans son container Docker.

La réponse fait ensuite le chemin inverse : l'entité créée est convertie en DTO de sortie, renvoyée avec le code 201 et son URL, et le composant Angular ferme la popup puis rafraîchit le journal — le badge de notifications se met à jour dans la foulée si une butée est concernée. Ce trajet est rigoureusement identique pour le desktop, puisque c'est le même front dans une fenêtre Electron, et pour le mobile, où Retrofit remplace le HttpClient d'Angular et l'intercepteur OkHttp celui du web. C'est toute la force de l'architecture centrée API : un seul chemin de données, trois clients.

10.5. Pourquoi REST, et pas autre chose ?

Le choix d'une API REST peut sembler aller de soi, mais il mérite d'être confronté aux alternatives contemporaines. **GraphQL** propose de laisser le client composer ses requêtes : séduisant quand les besoins d'affichage varient beaucoup d'un client à l'autre, mais superflu ici — les trois clients d'Hesias Fleet affichent les mêmes données selon les mêmes découpages, et les endpoints REST épousent déjà exactement les écrans. GraphQL aurait ajouté un schéma à maintenir, une couche de résolution

à sécuriser, et une complexité de cache, sans bénéfice tangible pour le projet. **gRPC**, de son côté, excelle dans la communication entre services à haute fréquence, mais son outillage navigateur reste contraignant, et le débogage binaire est moins accessible que du JSON lisible dans l'onglet Réseau des DevTools — un critère réel pour un projet où Swagger et les DevTools ont servi d'outils de travail quotidiens.

REST avec JSON offre enfin deux avantages décisifs dans un contexte de formation et d'évaluation : il est **universellement compris** — n'importe quel développeur, et n'importe quel membre du jury, peut lire le contrat dans Swagger et le tester en deux clics — et il est **outillé partout** : Retrofit côté Android, HttpClient côté Angular, tout l'écosystème le parle nativement. La règle que je retiens : choisir la technologie de communication la plus simple qui couvre le besoin, et documenter pourquoi les alternatives n'apportaient rien ici — c'est exactement l'objet de cette sous-section.

11. La base de données

Le modèle relationnel compte **onze tables** et découle directement de l'expression des besoins : un pôle « véhicules et journal », un pôle « magasin », et les utilisateurs qui relient les deux par la traçabilité. Le schéma est créé par la migration EF Core initiale et versionné avec le code — il n'existe aucun script SQL maintenu à la main.

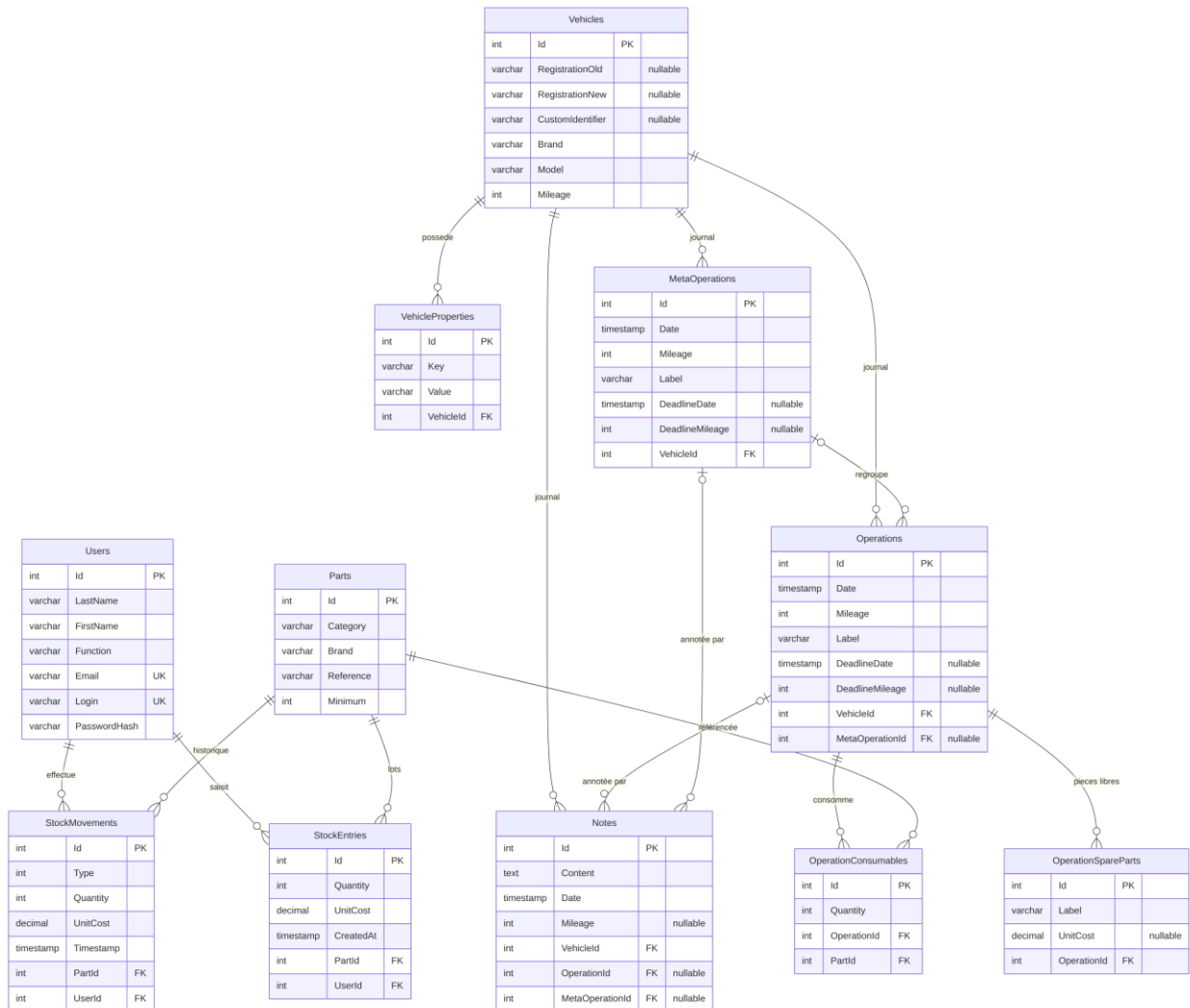


Figure 2 — Schéma de la base de données (diagramme entités-associations)

11.1. Lecture guidée du modèle

Commençons par le pôle « véhicules et journal ». La table **Vehicles** est le point d'ancrage : chaque véhicule y porte ses deux formats d'immatriculation et son identifiant libre — trois colonnes optionnelles, puisqu'un véhicule non immatriculé n'a que l'identifiant choisi par l'exploitant —, sa marque, son modèle et son kilométrage courant, celui que la solution externe met à jour. La table **VehicleProperties** lui adjoint des propriétés arbitraires sous forme de paires clé/valeur : c'est le bonus du sujet — type moteur, boîte de vitesses — modélisé sans figer de colonnes, si bien qu'un nouveau type de propriété ne demande aucune évolution du schéma.

Le journal proprement dit repose sur trois tables. **Operations** enregistre chaque intervention avec sa date, son kilométrage, son libellé, et ses deux butées optionnelles et indépendantes — calendrier et kilométrique ; une opération peut en outre référencer la méta-opération qui la regroupe. **MetaOperations** porte ces regroupements — la « Vidange moteur » et ses composantes — et peut recevoir elle-même une butée, conformément au sujet. **Notes** complète l'ensemble : chaque note appartient toujours à un véhicule, et peut en option se rattacher à une opération ou à une méta-opération — deux clés étrangères optionnelles qui matérialisent les trois types de notes du cahier des charges. Deux tables satellites relient enfin les opérations au reste du monde : **OperationConsumables** référence les pièces du magasin consommées, avec leur quantité — c'est sa saisie qui déclenche le retrait du stock —, tandis que **OperationSpareParts** accueille les pièces détachées saisies librement, hors stock, avec leur prix de revient optionnel.

Le pôle « magasin » tient en trois tables, et sa structure mérite qu'on s'y arrête car elle encode la règle métier la plus singulière du sujet. **Parts** est le référentiel des pièces : le triplet catégorie, marque et référence qui les identifie, et le seuil minimum d'alerte. **StockEntries** représente les lots : chaque entrée de stock y crée une ligne portant la quantité restante, le prix de revient unitaire et la date — et c'est précisément cette granularité « lot » qui rend le déstockage FIFO par prix trivial à implémenter et à auditer, là où une simple colonne de quantité l'aurait rendu impossible. **StockMovements**, enfin, est l'historique : chaque entrée, consommation ou ajustement y laisse une trace horodatée, signée par son utilisateur, avec la quantité signée et le prix du lot concerné. La table **Users** ferme le modèle : identité, fonction, email et login uniques, et le hash BCrypt du mot de passe — jamais le mot de passe lui-même.

11.2. Choix de conception notables

Cinq décisions structurent ce modèle. La première est la **séparation entre l'état et l'historique** du stock : **StockEntries** dit ce qu'il reste, **StockMovements** dit tout ce qui s'est passé. On peut ainsi reconstituer la vie complète d'une pièce — qui a entré quoi, qui a consommé quoi, à quel prix — sans que le calcul du disponible n'ait à parcourir tout l'historique. La deuxième est le choix de **butées optionnelles et indépendantes** : deux colonnes nullable plutôt qu'une table dédiée, car une opération a zéro, une ou deux butées, jamais davantage — la forme la plus simple qui couvre exactement le besoin.

La troisième décision concerne les **suppressions** : lorsqu'un véhicule disparaît, son journal disparaît avec lui — suppression en cascade —, mais lorsqu'une opération est supprimée, la méta-opération qui la regroupait doit survivre : la clé étrangère correspondante est configurée en SET NULL, pour détacher plutôt que détruire. Ce comportement est défini dans le langage de la base, au niveau de la contrainte elle-même, et non laissé à la vigilance du code applicatif. La quatrième est la **précision**

monétaire : tous les montants sont déclarés en `numeric(10,2)` — jamais de flottants pour de l'argent, afin d'éviter les erreurs d'arrondi binaire. La cinquième est l'**unicité garantie par la base** : l'email et le login des utilisateurs portent chacun un index unique, si bien qu'aucun doublon ne peut exister même si une vérification applicative venait à être contournée. L'extrait suivant, tiré du `DbContext`, montre comment ces contraintes sont exprimées :

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Précision des montants monétaires
    modelBuilder.Entity<StockEntry>().Property(s => s.UnitCost).HasPrecision(10, 2);
    modelBuilder.Entity<StockMovement>().Property(s => s.UnitCost).HasPrecision(10, 2);
    modelBuilder.Entity<OperationSparePart>().Property(s => s.UnitCost).HasPrecision(10, 2);

    // Un email et un login uniques par utilisateur
    modelBuilder.Entity<User>().HasIndex(u => u.Email).IsUnique();
    modelBuilder.Entity<User>().HasIndex(u => u.Login).IsUnique();

    // Quand une opération est supprimée, on ne casse pas la méta-opération :
    // on détache l'opération plutôt que de supprimer en cascade
    modelBuilder.Entity<Operation>()
        .HasOne(o => o.MetaOperation)
        .WithMany(m => m.Operations)
        .HasForeignKey(o => o.MetaOperationId)
        .OnDelete(DeleteBehavior.SetNull);
}
```

Extrait de `AppDbContext.cs` — contraintes définies dans le langage de la base via EF Core

11.3. La migration : le schéma versionné avec le code

Le schéma n'a pas été écrit en SQL à la main : il est généré par la migration initiale du 15 juillet, produite depuis les entités C# par la commande de création de migration d'EF Core, puis appliquée par la commande de mise à jour de la base. La migration est elle-même du code C# versionné dans Git : tout poste qui clone le dépôt peut recréer une base strictement identique, et toute évolution future du modèle donnera lieu à une nouvelle migration, rejouable dans les deux sens grâce à ses méthodes de montée et de descente de version.

En inspectant le SQL produit, on retrouve les choix PostgreSQL concrets : des clés primaires auto-incrémentées déclarées comme colonnes `identity` — la forme standard recommandée par Npgsql, préférable aux anciennes séquences `serial` —, des textes dans le type `text` natif de PostgreSQL, des dates en `timestamp with time zone` stockées en UTC, et des montants en `numeric` à deux décimales. Les index d'unicité et les comportements de suppression décrits plus haut figurent dans la même migration : la base est complète et cohérente dès sa première création, sans script d'appoint.

11.4. Le dictionnaire de données, en bref

Sans dérouler ici les onze tables colonne par colonne, quatre d'entre elles concentrent l'essentiel des décisions. Dans **Vehicles**, tout repose sur la souplesse d'identification : trois colonnes optionnelles — les deux formats d'immatriculation et l'identifiant libre — autour d'un noyau obligatoire `marque`, `modèle` et `kilométrage` entier. Dans **Operations**, ce sont les colonnes optionnelles de butée — une date et un entier de kilométrage, indépendants — et la clé étrangère optionnelle vers la méta-opération, en `SET NULL`, qui portent le sens ; la clé vers le véhicule est en revanche obligatoire et en cascade. Dans **StockEntries**, chaque lot conserve sa quantité restante — décrémentée par le FIFO —,

son prix unitaire en `numeric(10,2)`, sa date de création qui départage les lots à prix égal, et l'utilisateur qui l'a saisi. Dans **StockMovements** enfin, le type de mouvement est un entier d'énumération — entrée, consommation, ajustement —, la quantité est signée — positive pour un ajout, négative pour un retrait —, et chaque ligne porte son horodatage et son utilisateur : c'est la matérialisation directe de la règle de traçabilité RG-M9.

11.5. Volumétrie et comportement attendu

Les ordres de grandeur visés sont ceux d'une flotte d'entreprise réaliste : quelques centaines de véhicules, quelques milliers de pièces référencées, et un journal qui croît de quelques dizaines d'opérations par jour — soit des dizaines de milliers de lignes par an dans les tables d'opérations et de mouvements. À cette échelle, PostgreSQL est très à l'aise : les accès se font par clé primaire ou par clé étrangère indexée — le journal d'un véhicule, le stock d'une pièce — et les jointures du calcul de notifications restent bornées par la taille de la flotte.

Deux choix de conception préparent la montée en volume. L'historique des mouvements est en **append-only** : on n'y fait jamais de mise à jour, seulement des insertions et des lectures filtrées, ce qui est le régime le plus favorable pour un SGBD. Et les agrégats coûteux — la quantité disponible d'une pièce — se calculent sur les lots restants plutôt que sur l'historique complet : le nombre de lots actifs d'une pièce reste petit, car les lots vidés ne participent plus aux consommations. Si la flotte devenait très importante, les axes d'optimisation identifiés — pagination des listes, index complémentaires, projections ciblées — sont décrits en section 21.

12. L'API REST

12.1. Principes

L'API suit les conventions REST. Les **ressources** — véhicules, pièces, opérations, notes — sont identifiées par des URL stables et hiérarchiques : la collection des véhicules, un véhicule précis désigné par son identifiant, le journal d'un véhicule. Les **verbes HTTP** expriment l'action : GET pour lire, POST pour créer, PUT pour remplacer une ressource entière, PATCH pour une mise à jour partielle — c'est le verbe retenu pour le kilométrage —, DELETE pour supprimer. Les échanges se font en **JSON**, et les **codes de statut** portent le résultat : 200 pour une lecture réussie, 201 pour une création, 204 pour une suppression, 400 pour une requête invalide, 401 pour un défaut d'authentification, 404 pour une ressource inconnue.

Deux principes complètent ces conventions. L'API est **sans état** : chaque requête embarque son jeton d'authentification, aucune session n'est conservée côté serveur — c'est précisément ce qui permet aux trois clients, y compris le mobile natif, d'utiliser le même backend sans mécanique de cookies. Et les controllers ne manipulent jamais les entités de la base dans les échanges : des **DTOs** dédiés définissent le contrat d'entrée et de sortie, un point suffisamment important pour mériter sa propre sous-section, la 12.4.

12.2. L'authentification par JWT

Le client s'authentifie une seule fois, en envoyant son login et son mot de passe au point d'entrée de connexion. En retour, il reçoit un **JSON Web Token** signé en HMAC-SHA256, valable deux heures, qui contient l'identité de l'utilisateur — identifiant, login, email — dans ses claims. Toutes les requêtes suivantes portent ce jeton dans l'en-tête Authorization, et le middleware JWT d'ASP.NET Core valide la signature, l'émetteur, l'audience et l'expiration avant même que le controller ne s'exécute. Le code du controller de connexion illustre deux précautions de sécurité délibérées : la vérification BCrypt du mot de passe contre le hash stocké, et la réponse strictement identique que le login soit inconnu ou le mot de passe erroné — un attaquant n'apprend rien sur l'existence des comptes.

```
// POST api/auth/login
[HttpPost("login")]
public async Task<ActionResult<AuthResponseDto>> Login(LoginDto dto)
{
    var user = await _userRepository.GetByLoginAsync(dto.Login);

    // Utilisateur inexistant OU mot de passe incorrect : même réponse
    // (on ne révèle pas lequel des deux, par sécurité)
    if (user is null || !BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash))
        return Unauthorized("Login ou mot de passe incorrect.");

    var token = _tokenService.GenerateToken(user);
    return Ok(new AuthResponseDto { Token = token, User = ... });
}
```

AuthController.cs — connexion et délivrance du jeton

12.3. Le tour des endpoints, controller par controller

L'API expose huit controllers, une trentaine d'endpoints en tout. Plutôt qu'une liste brute, en voici la visite guidée, dans l'ordre où un client les rencontre.

Le controller d'**authentification** n'a qu'un seul point d'entrée, la connexion : il reçoit le couple login et mot de passe, et renvoie le jeton accompagné du profil de l'utilisateur. C'est le seul endpoint accessible sans jeton. Le controller des **utilisateurs** offre la gestion classique du référentiel : lister les utilisateurs, en consulter un, en créer un — le mot de passe fourni est haché avant toute persistance — et en supprimer un.

Le controller des **véhicules** couvre le cycle de vie complet : liste de la flotte, détail d'un véhicule avec ses propriétés, création, édition complète en PUT, et suppression. S'y ajoute l'endpoint le plus singulier de l'API : la mise à jour partielle du kilométrage en PATCH, sur une URL dédiée. Ce point d'entrée matérialise l'hypothèse du sujet selon laquelle une solution externe — télémetrie ou saisie régulière — vient actualiser le kilométrage en base : il ne modifie que cette donnée, et son existence même documente le contrat passé avec cette solution externe.

Le controller des **pièces** est le plus riche, car il porte tout le magasin. On y trouve d'abord le référentiel : liste des pièces avec leurs quantités disponibles, détail, création, édition — le fameux bouton de correction des dénominations — et suppression. Trois endpoints d'**autocomplétion** renvoient respectivement les catégories, les marques et les références déjà connues, pour alimenter les champs de saisie du front. Viennent enfin les quatre endpoints de **stock**, tous rattachés à une pièce donnée : la consultation de la situation — lots et alerte éventuelle —, l'entrée de stock avec quantité et prix de revient, la consommation qui applique le déstockage FIFO, et l'ajustement à la quantité réellement constatée.

Le controller des **opérations** articule le journal : liste générale, journal d'un véhicule donné, détail d'une opération, création — celle qui orchestre la consommation du stock, décrite pas à pas en section 10.4 — et suppression. Il expose aussi l'endpoint des **butées** d'un véhicule, qui renvoie pour chaque opération concernée l'échéance sous ses deux formes, en incrément et en absolu, avec l'indicateur « due ». Le controller des **méta-opérations** suit le même schéma — liste, par véhicule, détail, création avec ses composantes, suppression qui détache les opérations au lieu de les détruire —, et celui des **notes** gère les trois formes de notes du sujet : par véhicule, détail, création libre ou rattachée, suppression.

Reste le controller des **notifications**, qui n'expose qu'une lecture : la liste des butées atteintes ou dépassées sur l'ensemble de la flotte, triée par urgence — les retards les plus importants en tête. C'est cette unique source que consomment à la fois le badge de la barre supérieure et le tableau « Opérations à effectuer » du Dashboard. L'ensemble de ces endpoints est documenté par **Swagger**, accessible en développement : chaque point d'entrée y est décrit avec ses DTOs, et le bouton « Authorize » permet d'y coller un jeton pour tester les endpoints protégés depuis le navigateur — l'outil de travail quotidien de toute la phase backend.

12.4. Les DTOs : le contrat explicite de l'API

Chaque ressource possède ses DTOs d'entrée et de sortie, distincts des entités EF Core. L'exemple des opérations montre bien l'intérêt du découplage : pour créer une opération, le client n'envoie que ce qu'il doit fournir — l'identifiant du véhicule, la date, le kilométrage, le libellé, les butées

éventuelles, la liste des consommables réduite à des couples pièce/quantité, et les pièces libres avec leur prix optionnel. Ni identifiant d'opération, ni objets de navigation, ni aucune donnée que le serveur peut déterminer lui-même. En sortie, le DTO d'opération expose le journal tel que les clients l'affichent, et rien de plus.

Ce découplage a trois bénéfices concrets. Le premier est la **sécurité** : il est structurellement impossible d'exposer le hash d'un mot de passe ou de laisser un client fixer un identifiant, puisque ces champs n'existent pas dans les DTOs. Le deuxième est la **stabilité du contrat** : le modèle interne peut évoluer — une colonne renommée, une relation ajoutée — sans casser les trois clients, tant que les DTOs sont préservés. Le troisième est la **lisibilité de la documentation** : Swagger décrit exactement ces DTOs, ni plus ni moins. Côté clients, les interfaces TypeScript du web et les data classes Kotlin du mobile sont des miroirs de ces DTOs : tout écart de contrat devient une erreur de compilation, et non un bug découvert à l'exécution.

12.5. Conventions de réponse et gestion des erreurs

L'API applique des conventions uniformes que les trois clients exploitent de la même manière. Une lecture ou une mise à jour réussie renvoie 200 avec les données ; une création renvoie 201 avec la ressource créée et l'URL où la retrouver — les clients ferment alors la fenêtre de saisie et rafraîchissent la liste ; une suppression renvoie 204, sans corps. Un DTO invalide est intercepté par la validation d'ASP.NET Core et produit un 400 que les formulaires traduisent en messages de champ. Un jeton absent, invalide ou expiré produit un 401, traité de façon centralisée par les intercepteurs : déconnexion propre et retour à l'écran de connexion. Une ressource inconnue produit un 404, et une règle métier violée — le cas typique étant le stock insuffisant — produit une erreur explicite portant un message exploitable.

Un choix d'implémentation mérite d'être souligné : les règles métier refusées ne lèvent pas d'exceptions non gérées. Les services renvoient un résultat d'échec — un booléen ou une valeur nulle — que le controller traduit en code HTTP et en message. Le client peut ainsi afficher une erreur utile, comme « Stock insuffisant ou pièce introuvable », plutôt qu'une erreur serveur générique ; et les exceptions restent réservées aux situations réellement exceptionnelles.

12.6. Deux échanges commentés

Pour rendre le contrat concret, décrivons deux échanges tels qu'on les observe dans Swagger ou dans l'onglet Réseau du navigateur. Le premier est la **connexion**. Le client envoie une requête POST sur le point d'entrée de connexion, avec un corps JSON de deux champs : le login et le mot de passe. Si les identifiants sont corrects, la réponse 200 contient deux choses : le jeton — une chaîne compacte en trois segments séparés par des points, encodant l'en-tête de signature, les claims d'identité et la signature elle-même — et le profil de l'utilisateur : identifiant, nom, prénom, fonction, email, login. On remarque ce qui ne s'y trouve pas : ni le mot de passe, ni son hash, qui ne quittent jamais le serveur. Si les identifiants sont incorrects, la réponse 401 porte le message unique décrit plus haut, sans indice sur la cause exacte.

Le second échange est la **création d'une opération avec consommable et butée** — l'échange le plus riche de l'application. La requête POST porte l'en-tête d'autorisation avec le jeton, et un corps JSON qui rassemble tout ce que l'utilisateur a saisi dans la popup : l'identifiant du véhicule, la date en UTC, le kilométrage, le libellé — « Remplacement filtre à huile » —, la butée kilométrique à vingt mille

kilomètres de là, la liste des consommables réduite à des couples pièce/quantité, et une pièce libre avec son prix de revient. La réponse 201 renvoie l'opération créée avec son identifiant attribué par le serveur, et son URL dans l'en-tête de localisation. Derrière cette unique requête, on l'a vu en section 10.4, le serveur a consommé le stock lot par lot, tracé les mouvements, et enregistré l'opération : le client n'orchestre rien, il décrit une intention et reçoit un résultat.

13. Le frontend web en détail

13.1. Organisation du code

Le projet Angular sépare strictement le noyau technique des fonctionnalités. Le dossier **core** regroupe tout ce qui est transversal : le **guard** qui protège les routes, l'intercepteur qui gère l'authentification des requêtes, les modèles TypeScript — miroirs des DTOs de l'API — et un service HTTP par ressource : authentification, véhicules, pièces, opérations, notes, notifications. Le dossier **features** contient les écrans, regroupés par domaine fonctionnel : l'écran de connexion, le tableau de bord, les pages et fenêtres des véhicules — liste, fiche, popups de véhicule et de note —, celles du magasin — liste des pièces, popups de pièce et de mouvement de stock — et la popup de saisie des opérations et méta-opérations.

Cette séparation n'est pas cosmétique : elle matérialise une règle de dépendance. Les écrans dépendent du noyau — ils consomment ses services et ses modèles — mais jamais l'inverse, et aucun écran ne dépend d'un autre écran. Chaque page est par ailleurs un **composant standalone** chargé à la demande par le routeur : le bundle initial reste léger, chaque écran n'embarquant que ses propres dépendances Material. Quant aux modèles TypeScript, leur alignement sur les DTOs fait du compilateur un allié : si le contrat de l'API change, le front cesse de compiler au lieu d'échouer silencieusement à l'exécution.

13.2. La gestion d'état avec les signals

L'état de l'interface repose sur les **signals** d'Angular 19 plutôt que sur des Subjects RxJS. Le principe : un service expose une valeur observable en lecture seule, les composants la lisent directement dans leurs templates, et toute mise à jour redessine automatiquement les vues concernées — sans abonnements à gérer ni fuites à craindre. Le service de notifications en est l'exemple le plus parlant : il interroge l'API et publie la liste des butées dues dans un signal unique, que consomment simultanément le badge de la barre supérieure et le tableau du Dashboard. Une seule source de vérité, deux affichages toujours synchronisés, et un code de service qui tient en une vingtaine de lignes.

Les listes des écrans suivent le même patron : la liste des véhicules, celle des pièces avec leurs stocks, le journal d'une fiche sont des signals alimentés par les services HTTP, rafraîchis à la fermeture des popups de saisie. La détection de changements devient ainsi prévisible : une vue ne se redessine que lorsque l'une des valeurs qu'elle lit a effectivement changé.

13.3. La protection des routes et la gestion du jeton

Toutes les pages hormis la connexion sont placées derrière un **guard** fonctionnel : si aucun jeton n'est présent, la navigation est redirigée vers l'écran de connexion. La configuration du routeur reflète la structure de l'application : la route de connexion d'un côté, et de l'autre un groupe de routes protégées — tableau de bord, véhicules, fiche d'un véhicule, magasin — toutes chargées paresseusement, avec une redirection par défaut vers le Dashboard et une route joker qui rabat les URL inconnues vers l'accueil.

Le **guard** couvre l'utilisateur jamais connecté ; l'**intercepteur** — présenté avec son code en section 17 — couvre l'autre cas, celui du jeton expiré en cours de session : il ajoute l'en-tête d'autorisation à

chaque requête sortante et, si l'API répond 401, déconnecte proprement l'utilisateur et le ramène à l'écran de connexion. Entre les deux, aucun composant de l'application n'a jamais à se préoccuper d'authentification : c'est une préoccupation transversale, traitée une seule fois, au bon endroit.

13.4. La déclinaison desktop avec Electron

Le processus principal Electron crée une fenêtre de 1400 par 900 pixels — avec un minimum de 1024 par 700, en dessous duquel les tableaux de l'application deviendraient illisibles — et charge, selon l'environnement, le serveur de développement Angular ou le build de production embarqué. Les précautions de la checklist de sécurité officielle d'Electron sont appliquées : l'**isolation de contexte** est activée, si bien que le code de la page n'a aucun accès direct aux API de Node ; l'**intégration Node** est désactivée ; le script de préchargement n'expose qu'une information anodine — la plateforme du système ; et toute tentative d'ouvrir une nouvelle fenêtre est interceptée pour rediriger les liens externes vers le navigateur système plutôt que de les charger dans l'application.

Le résultat est une application Windows installable — l'installateur NSIS est produit par electron-builder en une commande — qui offre exactement les mêmes fonctionnalités que le web, pour un coût de développement marginal : la définition de la fenêtre, le packaging et le durcissement tiennent en une centaine de lignes.

13.5. Les formulaires réactifs et la validation

Tous les formulaires de l'application — connexion, véhicule, pièce, opération, mouvement de stock, note — utilisent les **formulaires réactifs** d'Angular : le modèle du formulaire est déclaré dans le composant avec ses validateurs, et le template ne fait que s'y lier. Les erreurs s'affichent au moment pertinent — quand le champ a été touché —, le bouton de validation reste désactivé tant que le formulaire est invalide, et l'API revalide de toute façon côté serveur : la validation du front est un confort d'usage, jamais une garantie de sécurité.

Les formulaires riches ajoutent leurs règles propres : kilométrages et quantités bornés à des valeurs positives, champs de butée optionnels mais cohérents, et surtout les **valeurs par défaut injectées à l'ouverture** des popups d'opération — la date et le kilométrage de la saisie précédente, matérialisation directe de la règle d'assistance à la saisie RG-V12. C'est un détail d'implémentation, mais c'est lui qui fait la différence entre une application que les techniciens acceptent d'utiliser et une application qu'ils contournent.

13.6. L'identité visuelle dans le code

L'identité décrite en section 6 se traduit techniquement par le système de thème de Material 3 : la palette verte et le charbon de la barre latérale sont déclarés une fois dans la feuille de style globale, et les composants — boutons, onglets, badges, tableaux — en dérivent leurs couleurs au travers des variables de thème, sans qu'aucun composant ne code une couleur en dur. Les états d'alerte utilisent la variable d'erreur du thème, si bien que le rouge des indicateurs du Dashboard, celui des lignes de butées dépassées et celui des messages de formulaire sont rigoureusement le même — la cohérence n'est pas une discipline, c'est une conséquence de l'architecture des styles.

Ce choix a une vertu très concrète, vérifiée lors du restylage du 17 août : appliquer l'identité complète à une application déjà fonctionnelle n'a demandé de retoucher pratiquement que la définition du thème et quelques styles de structure — la barre latérale, les cartes du Dashboard —

sans rouvrir les écrans un par un. Et le desktop en a hérité automatiquement, puisque c'est le même build.

14. L'application mobile en détail

14.1. Architecture MVVM

L'application Android est organisée en trois couches. La couche **data** regroupe les modèles — data classes Kotlin alignées sur les DTOs de l'API —, le client Retrofit et la gestion de session. La couche **ui** contient un couple Activity / ViewModel par écran : connexion, liste des véhicules, détail avec le journal, saisie d'opération. La couche **util** héberge les utilitaires transverses, notamment le formatage des dates. Les ViewModels portent l'état et les appels réseau, exécutés en coroutines ; les Activities ne font qu'observer et afficher — c'est le patron MVVM recommandé par Google, appliqué sans variante exotique.

14.2. Le contrat REST typé avec Retrofit

L'interface FleetApi déclare un endpoint par méthode : Retrofit en génère l'implémentation à l'exécution, et Gson assure la sérialisation JSON dans les deux sens. Le contrat est ainsi vérifié à la compilation, exactement comme côté Angular avec TypeScript — même philosophie, autre langage. L'extrait ci-dessous montre la correspondance directe entre chaque méthode Kotlin et le controller ASP.NET qu'elle appelle :

```
/**
 * Contrat REST de l'API Hesias Fleet.
 * Chaque méthode correspond à un endpoint d'un controller ASP.NET Core.
 */
interface FleetApi {

    // AuthController – POST api/auth/login
    @POST("auth/login")
    suspend fun login(@Body body: LoginRequest): AuthResponse

    // VehiclesController – GET api/vehicles
    @GET("vehicles")
    suspend fun getVehicles(): List<Vehicle>

    // OperationsController
    @GET("operations/vehicle/{vehicleId}")
    suspend fun getOperationsByVehicle(@Path("vehicleId") vehicleId: Int): List<Operation>

    @POST("operations")
    suspend fun createOperation(@Body body: CreateOperation): Operation

    // NotesController
    @GET("notes/vehicle/{vehicleId}")
    suspend fun getNotesByVehicle(@Path("vehicleId") vehicleId: Int): List<Note>
}
```

FleetApi.kt — le contrat REST de l'application mobile (extrait)

Un intercepteur OkHttp ajoute le Bearer token à chaque requête — le pendant de l'intercepteur Angular — et l'URL de base de l'API est fournie par la configuration de build : en développement, c'est l'adresse spéciale 10.0.2.2, par laquelle l'émulateur Android atteint la machine hôte ; en production, elle serait remplacée par le domaine réel, sans toucher au code.

14.3. Le journal unifié

L'écran de détail d'un véhicule fusionne trois sources — opérations, méta-opérations, notes — en une seule liste chronologique, comme sur le web. Le ViewModel lance les trois appels réseau en parallèle grâce aux coroutines, assemble des entrées de journal typées, puis l'adapter de la liste affiche chaque type avec sa mise en forme propre : une opération montre son kilométrage et ses consommables, une méta-opération ses composantes, une note son texte. Le technicien voit ainsi d'un seul coup d'œil l'historique complet du véhicule et les remarques laissées par ses collègues avant d'intervenir — c'est précisément le cas d'usage que le sujet assigne à la consultation mobile.

14.4. L'identification par QR code

Depuis la liste des véhicules, un bouton lance le scanner ZXing : le QR code — collé sous le capot, comme le suggère le sujet — contient l'identifiant du véhicule, et l'application ouvre directement son journal. Parmi les trois variantes de reconnaissance proposées en bonus, le QR code l'a emporté sur la lecture de plaque — peu fiable en atelier, où l'éclairage et les angles de prise de vue sont défavorables — et sur le NFC — qui aurait exigé d'équiper physiquement chaque véhicule d'un tag, là où un autocollant imprimé suffit. C'est un exemple de bonus traité en fin de projet, une fois l'essentiel livré, conformément à la discipline de backlog décrite en section 5.

14.5. La saisie minimale

Le formulaire de saisie d'opération applique la philosophie du sujet — réduire le besoin de saisie — dans chaque détail : la date du jour est pré-remplie, le dernier kilométrage connu du véhicule est proposé par défaut, et les consommables se choisissent dans une liste alimentée par le stock du magasin plutôt que d'être saisis au clavier. Une opération courante s'enregistre en quelques secondes, d'une main, debout à côté du véhicule. Côté serveur, l'API applique ensuite les mêmes règles que pour une saisie web — déstockage FIFO, traçabilité — puisque c'est le même endpoint : le mobile n'est jamais un chemin de contournement.

14.6. La gestion de session

Le gestionnaire de session centralise l'état de connexion : stockage du jeton et de l'utilisateur courant après l'authentification, mise à disposition pour l'intercepteur, purge à la déconnexion. Le stockage repose sur les EncryptedSharedPreferences d'AndroidX — présentées côté sécurité en section 17 — si bien que l'utilisateur reste connecté entre deux lancements de l'application sans que le jeton ne soit jamais écrit en clair sur l'appareil. Et si l'API répond 401 parce que le jeton a expiré, l'application purge la session et ramène l'utilisateur à l'écran de connexion : c'est le comportement exact du front web, transposé en Kotlin.

Ce parallélisme systématique entre les clients — intercepteur contre intercepteur, gestion du 401 identique, modèles alignés sur les mêmes DTOs — est un choix délibéré de conception : en maintenance, un développeur qui connaît l'un des clients comprend immédiatement les deux autres.

15. Organisation de l'interface : pourquoi ces écrans ?

L'interface n'est pas un empilement d'écrans : chaque onglet, chaque fenêtre et chaque bouton répond à un besoin précis du cahier des charges. Cette section explique ces choix un par un — car savoir justifier une interface est aussi important que savoir la construire.

15.1. La structure générale : une sidebar à trois entrées

L'application web s'organise autour d'une barre latérale fixe qui ne propose que trois entrées : **Dashboard**, **Véhicules** et **Magasin**. C'est volontairement minimal. L'utilisateur cible — technicien, responsable magasin — n'est pas un informaticien et doit trouver sa destination en une seconde ; chaque entrée supplémentaire dans un menu se paie en hésitation. Les opérations, méta-opérations et notes n'ont pas de rubrique propre, et ce n'est pas un oubli : elles n'existent qu'au sein du journal d'un véhicule, et les promouvoir au premier niveau aurait créé des écrans orphelins — une liste de toutes les opérations de la flotte, sans contexte, ne répond à aucune question qu'un utilisateur se pose réellement. Une barre supérieure complète la structure avec le titre de la page courante et deux éléments permanents : le badge de notifications et le menu utilisateur avec la déconnexion.

15.2. Le Dashboard comme écran d'accueil

Après connexion, l'utilisateur arrive sur le Dashboard, et non sur une liste. La raison est fonctionnelle : le sujet impose de notifier les opérations arrivées en butée et les alertes de stock, et une notification qui exige une action pour être vue n'en est pas une. Le Dashboard affiche donc trois indicateurs — le nombre de véhicules, les opérations dues, les alertes de stock, ces deux derniers passant en rouge dès qu'ils sont non nuls — puis deux tableaux : « Opérations à effectuer », où chaque ligne donne le véhicule concerné avec un lien direct vers sa fiche et l'échéance exprimée en date et en kilomètres, et « Alertes de stock », avec un accès direct au magasin. Le badge de la barre supérieure affiche le même compteur en permanence, sur tous les écrans : c'est le système de notification retenu — sobre, toujours visible, jamais intrusif.

15.3. La fiche véhicule en trois onglets

La fiche d'un véhicule concentre des informations de natures très différentes ; plutôt qu'une page interminable, elle est découpée en trois onglets correspondant à trois intentions distinctes de l'utilisateur. L'onglet **Journal** présente l'historique chronologique mêlant opérations, méta-opérations — sous forme de panneaux dépliables qui listent leurs composantes — et notes ; c'est l'onglet par défaut, car « que s'est-il passé sur ce véhicule ? » est de loin la question la plus fréquente. L'onglet **Butées** donne la situation des échéances, en incrément et en absolu comme l'exige le sujet, dans un tableau dédié où les lignes dues sont mises en évidence — l'utilisateur qui vient planifier des interventions n'a pas à reconstituer cette information depuis le journal. L'onglet **Fiche**, enfin, porte l'identité du véhicule : immatriculations, identifiant libre, et les propriétés arbitraires clé/valeur comme le type moteur — des données consultées rarement, mais indispensables au moment de commander la bonne pièce.

15.4. Des popups plutôt que des pages pour la saisie

Toutes les créations et éditions — véhicule, pièce, opération, méta-opération, note, mouvement de stock — passent par des fenêtres de dialogue et non par des pages dédiées. Ce choix a deux justifications. La première est la préservation du contexte : l'utilisateur saisit une opération en gardant le journal sous les yeux, et la liste se rafraîchit à la fermeture de la fenêtre — il n'est jamais téléporté ailleurs dans l'application. La seconde tient à la saisie en série, sur laquelle le sujet insiste : après une saisie, la popup se rouvre instantanément, pré-remplie avec la date et le kilométrage de la saisie précédente — l'assistance à la saisie de la règle RG-V12 — si bien qu'enchaîner cinq opérations sur le même véhicule ne coûte que les cinq libellés.

15.5. Le magasin : autocomplétion et bouton d'édition

L'écran Magasin liste les pièces avec leur stock disponible et leur seuil d'alerte. Le formulaire de création utilise trois champs à autocomplétion — catégorie, marque, référence — alimentés par les endpoints dédiés de l'API : l'application propose ce qu'elle connaît déjà tout en acceptant de nouvelles valeurs, exactement le comportement décrit par le cahier des charges, qui limite à la fois les fautes de frappe et la création de doublons involontaires. Chaque ligne de la liste porte un bouton d'édition — la réponse au « bouton d'édition pour corriger la dénomination » du sujet — et un bouton de mouvement de stock, qui ouvre une popup à trois modes : entrée avec quantité et prix de revient, consommation, et ajustement à la quantité réellement constatée après un inventaire.

15.6. L'application mobile : le parcours du technicien

L'application Android reprend le strict nécessaire pour la saisie sur site, dans un enchaînement pensé pour l'atelier. La connexion utilise les mêmes identifiants que le web, et le jeton conservé chiffré évite de se reconnecter à chaque lancement. La liste des véhicules se parcourt par recherche rapide, ou se court-circuite entièrement par le scan du QR code. Le journal du véhicule se consulte avant d'intervenir, pour lire les notes des collègues. La saisie d'opération, enfin, est réduite à sa plus simple expression — date et kilométrage pré-remplis, consommables choisis dans le stock. Quatre écrans, pas un de plus : tout ce qui relève de la gestion — référentiels, seuils, corrections — reste sur le web et le desktop, là où l'on dispose d'un clavier et d'un grand écran.

16. Le fonctionnement du site

Le diagramme ci-dessous synthétise la navigation dans l'application web et desktop, de la connexion aux différentes fenêtres de saisie. Les sections qui suivent le parcourent en mots, du point de vue de l'utilisateur puis de chaque profil.

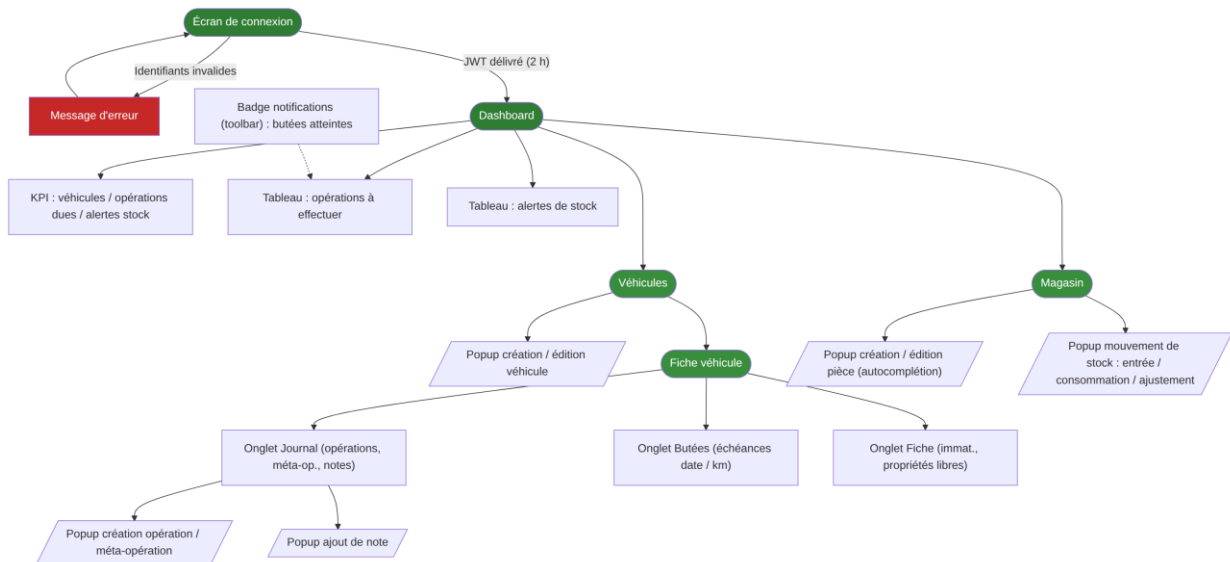


Figure 3 — Fonctionnement du site : navigation et fenêtres

16.1. Le parcours type

La connexion. L'utilisateur saisit son login et son mot de passe. En cas d'identifiants invalides, l'API répond 401 et un message d'erreur s'affiche — identique quelle que soit la cause, login inconnu ou mot de passe erroné, pour ne rien révéler. En cas de succès, le jeton est conservé côté client et l'utilisateur est redirigé vers le Dashboard. Le guard de route protège toutes les pages : un accès direct à une URL interne sans jeton ramène à la connexion.

La vue d'ensemble. Le Dashboard présente les indicateurs et les deux tableaux d'alertes. L'intercepteur ajoute silencieusement le jeton à chaque appel ; si l'API répond 401 — jeton expiré après deux heures —, l'utilisateur est déconnecté proprement et ramené à l'écran de connexion, sans écran d'erreur brutal ni état incohérent.

Le travail sur un véhicule. Depuis la liste des véhicules — où création et édition se font en popup —, l'utilisateur ouvre une fiche. Il consulte le journal, déploie une méta-opération pour voir ses composantes, ajoute une opération via la popup : date et kilométrage pré-remplis, consommables choisis dans le stock — leur saisie déclenche le déstockage FIFO côté API —, pièces libres ajoutées à la volée, butée optionnelle en date et en kilomètres. L'onglet Butées lui montre à tout moment ce qui arrive à échéance, sous les deux formes exigées par le sujet.

La gestion du magasin. Dans l'écran Magasin, il crée des pièces avec l'aide de l'autocomplétion, corrige une dénomination par le bouton d'édition, enregistre des entrées de stock avec leur prix de revient, ou ajuste une quantité après un inventaire. Toutes ces actions sont tracées en base avec l'utilisateur et l'horodatage — sans action supplémentaire de sa part.

La notification. À chaque chargement, le badge de la barre supérieure interroge l'endpoint des notifications : si des butées sont atteintes ou dépassées, le compteur s'affiche, et le détail attend l'utilisateur sur le Dashboard, trié par urgence.

16.2. La prise en main par profil

Le responsable magasin vit principalement dans deux écrans : le Dashboard le matin — les alertes de stock lui indiquent immédiatement quoi recommander — et le Magasin le reste du temps, où il enregistre les réceptions, corrige les dénominations et ajuste les quantités après inventaire. Le tri par alerte et la recherche lui évitent de parcourir tout le référentiel.

Le gestionnaire de flotte part lui aussi du Dashboard, mais s'intéresse au tableau « Opérations à effectuer » : chaque ligne le mène en un clic à la fiche du véhicule concerné, onglet Butées, où il lit l'échéance en absolu et en incrément. Il planifie les interventions en conséquence, crée les nouveaux véhicules à leur arrivée dans la flotte, et complète leurs propriétés techniques dans l'onglet Fiche pour préparer le travail des techniciens.

Le technicien utilise surtout l'application mobile en atelier : scan du QR code sous le capot, coup d'œil au journal pour lire les notes laissées par les collègues, saisie des opérations au fil de l'intervention — l'application pré-remplit la date et le kilométrage, et décompte automatiquement les consommables utilisés. De retour au bureau, il retrouve exactement les mêmes données sur le web ou sur l'application Windows. Cette continuité entre profils et supports est le fil conducteur de l'interface : une même donnée, saisie une fois, visible partout, immédiatement.

16.3. Les cas particuliers et les états d'erreur

Un parcours utilisateur ne se juge pas qu'au cas nominal ; voici ce que voit l'utilisateur quand les choses se passent mal, et pourquoi. Si l'**API est indisponible** — container arrêté, réseau coupé —, les appels échouent et l'interface l'affiche sans se figer : les listes restent vides avec leur état dédié, et le badge de notifications retombe silencieusement à zéro plutôt que de bloquer la barre supérieure — un choix délibéré du service de notifications, qui traite l'erreur en publiant une liste vide. Si la **session expire** en cours d'utilisation, la première requête qui essuie un 401 déclenche la déconnexion propre : retour à l'écran de connexion, sans écran d'erreur technique ni état à moitié chargé.

Si une **saisie viole une règle métier** — le cas type étant la consommation d'un stock insuffisant —, la popup reste ouverte, le message explicite de l'API s'affiche, et rien n'a été modifié en base : l'utilisateur corrige et revalide, sans avoir à reconstituer sa saisie. Si une **validation de formulaire** échoue — champ requis manquant, kilométrage négatif —, l'erreur apparaît sous le champ concerné dès qu'il a été touché, et le bouton de validation reste inactif : l'erreur est montrée avant même d'être commise. Enfin, une **URL inconnue** — lien obsolète, faute de frappe — ramène à l'accueil par la route joker du routeur, et, en production, la configuration nginx garantit qu'un rechargement de page sur n'importe quelle route interne recharge bien l'application au lieu de produire une erreur du serveur.

17. Réalisations significatives (extraits de code)

Cette section présente les extraits de code les plus significatifs du projet, en les argumentant — y compris sous l'angle de la sécurité. Ils ont été choisis parce que chacun incarne une décision : une règle métier singulière, une préoccupation transversale traitée au bon endroit, ou une précaution de sécurité concrète. Le reste du code suit les mêmes principes ; ces cinq extraits en sont la démonstration.

17.1. Le déstockage FIFO par prix (StockService)

C'est la règle métier la plus spécifique du sujet : consommer en priorité les lots les moins chers. L'implémentation parcourt les lots triés par prix croissant puis par ancienneté, les vide dans l'ordre, et trace chaque retrait dans l'historique au prix du lot réellement consommé. La consommation est refusée en bloc si le stock total est insuffisant.

```
public async Task<bool> ConsumeStockAsync(int partId, int quantity, int? userId = null)
{
    var part = await _partRepository.GetByIdWithStockAsync(partId);
    if (part is null) return false;

    // Stock insuffisant : on refuse la consommation
    if (GetAvailableQuantity(part) < quantity) return false;

    var remaining = quantity;

    // Déstockage FIFO par PRIX : on vide d'abord les lots les moins chers
    var lots = part.StockEntries
        .Where(e => e.Quantity > 0)
        .OrderBy(e => e.UnitCost)
        .ThenBy(e => e.CreatedAt)
        .ToList();

    foreach (var lot in lots)
    {
        if (remaining <= 0) break;
        var taken = Math.Min(lot.Quantity, remaining);
        lot.Quantity -= taken;
        remaining -= taken;

        // Trace le mouvement (consommation, quantité négative)
        _context.StockMovements.Add(new StockMovement {
            PartId = partId, Type = StockMovementType.Consumption,
            Quantity = -taken, UnitCost = lot.UnitCost,
            Timestamp = DateTime.UtcNow, UserId = uid
        });
    }
    await _partRepository.SaveChangesAsync();
    return true;
}
```

StockService.cs — consommation FIFO par prix avec traçabilité

Trois points méritent l'attention. Le tri secondaire par date de création rend le comportement déterministe quand deux lots ont le même prix — sans lui, l'ordre de consommation dépendrait de détails d'implémentation de la base. La vérification préalable du disponible garantit qu'une

consommation échoue en bloc ou réussit en bloc : on ne se retrouve jamais avec un retrait partiel. Enfin, chaque retrait est enregistré au prix du lot réellement consommé, et non à un prix moyen : c'est ce qui permet de valoriser précisément le coût des opérations d'entretien — dans l'exemple du sujet, une consommation de quinze filtres produit deux mouvements, dix à 4,20 € et cinq à 5,60 €.

17.2. Le calcul des butées (DeadlineService)

Le service des butées transforme les données brutes — date cible, kilométrage cible — en information directement affichable : jours restants, kilomètres restants, et un indicateur « due » levé dès qu'au moins une des deux échéances est atteinte ou dépassée. C'est lui qui produit la double lecture exigée par le sujet, en incrément et en absolu.

```
public DeadlineStatus? Compute(Operation operation, int currentMileage, DateTime now)
{
    if (operation.DeadlineDate is null && operation.DeadlineMileage is null)
        return null; // pas de butée : rien à calculer

    var status = new DeadlineStatus { OperationId = operation.Id, Label = operation.Label };

    if (operation.DeadlineDate is not null) // butée calendaire
    {
        status.DeadlineDate = operation.DeadlineDate;
        status.DaysRemaining = (int)(operation.DeadlineDate.Value.Date - now.Date).TotalDays;
    }
    if (operation.DeadlineMileage is not null) // butée kilométrique
    {
        status.DeadlineMileage = operation.DeadlineMileage;
        status.KilometersRemaining = operation.DeadlineMileage.Value - currentMileage;
    }

    // "Due" si AU MOINS une des butées est atteinte ou dépassée
    var dueByDate = status.DaysRemaining is not null && status.DaysRemaining <= 0;
    var dueByMileage = status.KilometersRemaining is not null && status.KilometersRemaining <=
0;
    status.IsDue = dueByDate || dueByMileage;
    return status;
}
```

DeadlineService.cs — échéances en incrément et en absolu

Ce service illustre l'intérêt de la couche domaine : c'est du C# pur, sans la moindre dépendance — ni base de données, ni HTTP. Le calcul sur des dates tronquées à minuit évite les décalages d'un jour selon l'heure de la journée, et les valeurs restantes peuvent être négatives — c'est voulu : un retard de trois mille kilomètres s'affiche comme tel, et sert au tri par urgence des notifications. Le même service est appelé par deux endpoints — les butées d'un véhicule, et les notifications de toute la flotte — sans une ligne dupliquée.

17.3. L'orchestration métier : la création d'une opération

Le contrôler des opérations illustre le rôle de la couche API : il ne contient aucune règle métier, mais séquence les services dans le bon ordre — d'abord la consommation du stock pour chaque consommable demandé, qui peut échouer et interrompre la création avec un message exploitable, puis la construction de l'entité complète avec ses consommables et ses pièces libres.

```
// POST api/operations
[HttpPost]
public async Task<ActionResult<OperationDto>> Create(CreateOperationDto dto)
{
    // 1. Vérifier et retirer le stock pour chaque consommable
    foreach (var consumable in dto.Consumables)
    {
        var success = await _stockService.ConsumeStockAsync(
            consumable.PartId, consumable.Quantity);
        if (!success)
            return BadRequest($"Stock insuffisant ou pièce introuvable (PartId
{consumable.PartId}).");
    }

    // 2. Créer l'opération avec ses consommables et pièces libres
    var operation = new Operation
    {
        VehicleId = dto.VehicleId,
        Date = dto.Date,
        Mileage = dto.Mileage,
        Label = dto.Label,
        DeadlineDate = dto.DeadlineDate,
        DeadlineMileage = dto.DeadlineMileage,
        Consumables = dto.Consumables
            .Select(c => new OperationConsumable { PartId = c.PartId, Quantity = c.Quantity })
            .ToList(),
        SpareParts = dto.SpareParts
            .Select(sp => new OperationSparePart { Label = sp.Label, UnitCost = sp.UnitCost })
            .ToList()
    };

    var created = await _repository.CreateAsync(operation);
    return CreatedAtAction(nameof(GetById), new { id = created.Id }, ToDto(created));
}
```

OperationsController.cs — le contrôleur orchestre, les services décident

On lit dans cet extrait la philosophie de toute l'API : le contrôleur reçoit un DTO validé, appelle les services qui portent les règles, traduit leurs refus en réponses HTTP explicites, et convertit le résultat en DTO de sortie avec le code 201 et l'URL de la ressource créée. Les règles elles-mêmes — le FIFO, le refus du stock insuffisant — vivent ailleurs, dans des services réutilisables et testables.

17.4. Les intercepteurs : l'authentification traitée une seule fois

Côté web, un intercepteur unique centralise deux responsabilités : ajouter le Bearer token à chaque requête sortante, et déconnecter proprement l'utilisateur si l'API répond 401 — jeton expiré ou invalide. Aucun composant de l'application n'a donc à se soucier d'authentification.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
    const auth = inject(AuthService);
    const token = auth.token();

    const authorized = token
        ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
        : req;

    return next(authorized).pipe(
```



```

    catchError((err: HttpResponse) => {
        if (err.status === 401 && auth.isAuthenticated()) {
            auth.logout(); // jeton expiré : retour à l'écran de connexion
        }
        return throwError(() => err);
    })
};

```

auth.interceptor.ts — Bearer token et gestion centralisée du 401

L'application mobile applique le même patron avec un intercepteur OkHttp : chaque requête Retrofit part avec son en-tête d'autorisation, et le 401 déclenche la purge de session. Cette symétrie n'est pas un hasard mais une règle de conception : les préoccupations transversales — authentification, gestion des erreurs — sont traitées au même endroit logique dans chaque client.

17.5. Le stockage chiffré du jeton sur mobile

Sur Android, le jeton et l'utilisateur courant ne sont pas stockés en clair : le gestionnaire de session s'appuie sur les EncryptedSharedPreferences d'AndroidX, avec une clé maîtresse AES-256-GCM générée et conservée dans le Keystore matériel du téléphone.

```

val masterKey = MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .build()

EncryptedSharedPreferences.create(
    context, PREFS_NAME, masterKey,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
)

```

SessionManager.kt — stockage chiffré du JWT (AndroidX security-crypto)

L'alternative — des SharedPreferences classiques — aurait laissé le jeton lisible dans les fichiers de l'application sur un appareil compromis ou simplement prêté. Avec le chiffrement adossé au Keystore, les clés ne quittent jamais la puce sécurisée du téléphone : même une copie complète des fichiers de l'application ne livre pas le jeton. C'est une ligne de configuration de plus pour une classe de risques en moins — le genre d'arbitrage que la veille de sécurité, décrite en section 18, apprend à faire systématiquement.

18. Sécurité

18.1. Les mesures mises en œuvre

La sécurité du projet s'est construite autour des risques concrets d'une application de gestion multi-clients, traités un par un. Contre le **vol des mots de passe**, le hachage BCrypt avec son sel intégré et son facteur de coût rend l'attaque par force brute très onéreuse même si la base fuitait ; et le hash ne circule jamais, puisque aucun DTO de réponse ne le contient. Contre l'**usurpation de session**, le jeton JWT est signé en HMAC-SHA256 et validé intégralement à chaque requête — signature, émetteur, audience, expiration de deux heures — par le middleware, avant tout code applicatif. Contre l'**énumération de comptes**, la réponse de connexion est identique que le login existe ou non.

Contre l'**injection SQL**, aucune requête n'est concaténée : tout passe par EF Core et LINQ, donc par du SQL paramétré — et Angular, de son côté, échappe par défaut toute interpolation dans les templates, ce qui couvre le risque symétrique du XSS ; le projet n'utilise d'ailleurs jamais d'injection directe de HTML. Contre les **requêtes cross-origin indésirables**, la politique CORS de développement n'autorise que l'origine du front, et la production supprime le problème à la racine puisque le proxy nginx place front et API sous la même origine. Contre le **vol du jeton sur mobile**, le stockage chiffré présenté en 17.5. Contre les **failles propres à Electron**, l'isolation de contexte, la désactivation de l'intégration Node et l'ouverture des liens externes hors de l'application. Contre les **incohérences de comptes**, les index uniques sur l'email et le login au niveau de la base elle-même. Et contre l'**accès non authentifié**, la protection des endpoints par le middleware JWT côté serveur, doublée du guard de route côté client — la première étant la vraie barrière, le second un confort d'expérience.

18.2. La veille sur les vulnérabilités de sécurité

Tout au long du projet, j'ai maintenu une veille de sécurité centrée sur les technologies employées. Mes sources principales : l'OWASP — le Top 10 et les cheat sheets consacrées à l'authentification, aux JWT et au stockage des mots de passe —, la GitHub Advisory Database et les alertes de dépendances du dépôt, les notes de version de sécurité de Microsoft pour .NET et ASP.NET Core, le blog Angular pour les correctifs du framework, et l'outil npm audit pour les dépendances JavaScript. Les recherches types portaient sur les bonnes pratiques JWT, le facteur de coût de BCrypt, la sanitisation XSS d'Angular, la checklist de sécurité d'Electron et les risques d'injection avec EF Core.

Cette veille a directement influencé le code, et c'est ce qui la distingue d'une lecture passive : le choix de BCrypt plutôt qu'un simple SHA-256 vient de la cheat sheet OWASP sur le stockage des mots de passe ; la réponse d'authentification non révélatrice applique la recommandation contre l'énumération de comptes ; le durcissement d'Electron suit point par point la checklist officielle ; le stockage chiffré du jeton sur Android remplace la solution naïve des SharedPreferences en clair ; et la clé JWT a été externalisée dans la configuration, avec une valeur de développement explicitement marquée à remplacer — le dépôt ne contient aucun secret de production.

18.3. Une situation de travail ayant nécessité une recherche

Le besoin. En connectant le front Angular à l'API, les requêtes échouaient dans le navigateur avec des erreurs CORS — y compris les réponses d'erreur 401, qui arrivaient sans les en-têtes d'autorisation cross-origin. Le diagnostic était d'autant plus confus que le front ne voyait qu'une erreur réseau

générique là où l'API répondait en réalité un 401 parfaitement valide : le symptôme masquait la cause.

La recherche. J'ai effectué la recherche en anglais, avec des mots-clés du type « asp.net core cors 401 response missing access-control-allow-origin » puis « middleware order UseCors UseAuthentication ». Les résultats renvoyaient principalement vers la documentation officielle Microsoft Learn — la page consacrée à l'activation du CORS dans ASP.NET Core —, vers plusieurs questions Stack Overflow décrivant des symptômes identiques, et vers des issues du dépôt GitHub d'aspnetcore. J'ai retenu la documentation Microsoft comme source de référence, parce qu'elle est officielle et à jour pour .NET 8, et Stack Overflow comme source de confirmation, pour les cas concrets semblables au mien.

La solution. L'ordre des middlewares est déterminant : l'appel au CORS doit précéder l'authentification et l'autorisation dans le pipeline, faute de quoi les réponses d'erreur produites par ces dernières ne portent pas les en-têtes cross-origin. La correction — remonter l'appel UseCors avant UseAuthentication — est visible dans le fichier Program.cs du dépôt, accompagnée d'un commentaire qui explique la règle pour mon moi futur. Cette recherche m'a aussi fait comprendre le mécanisme des requêtes préliminaires OPTIONS envoyées par le navigateur, et pourquoi le proxy nginx de production supprime entièrement le problème : quand tout provient de la même origine, il n'y a plus de CORS du tout.

18.4. Le projet au prisme de l'OWASP Top 10

Pour structurer l'analyse, j'ai pris l'habitude de relire le projet au travers du Top 10 de l'OWASP, édition 2021 — l'exercice vaut la peine d'être restitué ici en quelques lignes. Sur le **contrôle d'accès** (A01), les endpoints sont protégés par le middleware JWT ; l'absence de rôles est un choix assumé du sujet, listé dans les axes d'amélioration. Sur les **défaillances cryptographiques** (A02), BCrypt, HMAC-SHA256 et AES-256-GCM couvrent respectivement les mots de passe, les jetons et le stockage mobile ; la généralisation du HTTPS en production reste à faire et figure elle aussi en axe d'amélioration. Sur l'**injection** (A03), le duo EF Core côté serveur et échappement Angular côté client ferme les deux portes classiques.

Sur la **conception non sécurisée** (A04), le principe directeur est que les règles métier ne s'appliquent que côté serveur — la validation du front n'est qu'un confort, et le refus du stock insuffisant en est l'exemple. Sur la **mauvaise configuration** (A05), Swagger n'est exposé qu'en développement, le CORS est restreint, Electron durci. Sur les **composants vulnérables** (A06), les versions sont récentes et verrouillées, et les alertes GitHub et npm audit ont été suivies pendant tout le projet. Sur l'**authentification** (A07), la réponse non révélatrice, l'expiration des jetons et le hachage coûteux sont en place ; la limitation du débit sur la connexion est identifiée comme complément souhaitable. Sur l'**intégrité** (A08), le build en deux étapes installe les versions exactes du fichier de verrouillage, et les migrations sont versionnées. Sur la **journalisation** (A09), la traçabilité métier des mouvements de stock est complète, la journalisation applicative structurée restant un axe d'amélioration. La **falsification de requêtes côté serveur** (A10), enfin, ne concerne pas le projet : l'API n'effectue aucune requête sortante vers des URL fournies par l'utilisateur.

19. Plan de tests et jeu d'essai

19.1. La stratégie de test

La validation du développement repose sur des scénarios de tests manuels exécutés à deux niveaux : d'abord au niveau de l'API via Swagger — chaque endpoint testé avec des données valides, invalides et limites, avec et sans jeton —, puis au niveau des interfaces des trois clients, en déroulant les parcours utilisateur complets. Dix-huit scénarios, numérotés T1 à T18, couvrent les règles de gestion de la section 4 ; chacun référence les règles qu'il valide, si bien que la couverture se vérifie par simple recoupement. En voici la substance, groupée par thème.

L'authentification (T1 à T5). Le premier scénario vérifie la connexion nominale : identifiants corrects, jeton reçu, redirection vers le Dashboard et chargement du badge de notifications. Le deuxième teste les deux échecs — login inconnu, puis mot de passe erroné — et vérifie que le message est strictement identique dans les deux cas et qu'aucun jeton n'est émis (RG-U4). Le troisième contrôle l'accès sans jeton : un appel direct à l'API sans en-tête d'autorisation reçoit un 401, et un accès direct à une URL interne du front ramène à la connexion. Le quatrième provoque l'expiration du jeton et vérifie la déconnexion propre. Le cinquième couvre la création d'utilisateur : le hash n'apparaît dans aucune réponse, et les doublons d'email ou de login sont refusés par la base (RG-U1, RG-U2).

Le magasin (T8 à T12, T18). Les scénarios de stock vérifient successivement : que deux entrées à prix différents créent bien deux lots distincts et deux mouvements horodatés et signés (RG-M5, RG-M9) ; que la consommation vide les lots dans l'ordre du FIFO par prix — c'est le jeu d'essai détaillé en 19.2 (RG-M6) ; qu'une consommation supérieure au disponible est refusée sans modifier ni les lots ni l'historique (RG-M7) ; qu'un seuil minimum porté au-dessus du disponible déclenche l'alerte sur le Dashboard et dans le magasin (RG-M4) ; que l'autocomplétion propose les valeurs connues et que le bouton d'édition corrige une dénomination (RG-M2, RG-M3) ; et qu'un ajustement à la baisse après une casse corrige la quantité en laissant sa trace (RG-M8).

Les véhicules et le journal (T6, T7, T13 à T16). On y vérifie le cycle de vie d'un véhicule — création avec immatriculation, édition, création d'un véhicule non immatriculé avec identifiant libre (RG-V1) ; la mise à jour du kilométrage par l'endpoint dédié, et sa répercussion immédiate sur la fiche et les butées (RG-V2) ; la création d'une opération avec consommable, qui doit à la fois apparaître au journal et décrémenter le stock avec son mouvement de consommation (RG-V4, RG-V5) ; la méta-opération « Vidange moteur » à trois composantes, avec sa butée portée par la méta-opération, et la survie de celle-ci à la suppression d'une composante (RG-V7, RG-V8) ; le passage d'une butée à échéance, qui doit lister l'opération comme « à effectuer », incrémenter le badge, et afficher l'échéance sous ses deux formes (RG-V6, RG-V10, RG-V11) ; et l'assistance à la saisie — deux opérations saisies à la suite, la seconde pré-remplie avec la date et le kilométrage de la première (RG-V12).

Le parcours croisé (T17). Le dernier scénario traverse les supports : connexion sur mobile, scan du QR code d'un véhicule, consultation du journal, saisie d'une opération — puis vérification immédiate, sur le web, que l'opération apparaît au journal et que le stock a été décompté. C'est le test de la promesse centrale de l'architecture : une seule source de données pour trois clients.

19.2. Le jeu d'essai : la consommation FIFO du stock

La fonctionnalité la plus représentative du projet est le déstockage FIFO par prix, car elle combine règle métier, persistance et traçabilité. Le jeu d'essai reprend l'exemple chiffré du cahier des charges. Données en entrée : la pièce « Filtre à huile – Purflux – LS923 », stock initialement vide ; une première entrée de dix unités à 4,20 € ; une seconde entrée de vingt unités à 5,60 € ; puis la saisie d'une opération « Remplacement filtre à huile » consommant quinze unités.

Vérification	Résultat attendu	Résultat obtenu
Stock disponible après les deux entrées	30 unités, en deux lots : 10 à 4,20 € et 20 à 5,60 €	Conforme
Lots après consommation de 15 unités	Lot à 4,20 € : 0 restant ; lot à 5,60 € : 15 restants	Conforme — le lot le moins cher est vidé en premier
Mouvements enregistrés	Deux consommations : -10 à 4,20 € et -5 à 5,60 €, horodatées et signées	Conforme
Consommation de 40 unités (stock : 15)	Refus explicite, stock et historique inchangés	Conforme
Seuil minimum porté à 20	Alerte de stock visible (Dashboard et badge)	Conforme

Ce jeu d'essai est rejouable à volonté : la base étant conteneurisée, une suppression du volume suivie d'un redémarrage et de la migration recrée un environnement vierge en moins d'une minute, et la collection de requêtes conservée dans Rider ressaisit les données de départ.

19.3. Analyse des écarts et non-régression

Aucun écart n'a été constaté entre les résultats attendus et obtenus sur le jeu d'essai final. Deux écarts avaient en revanche été détectés par ces mêmes scénarios en cours de développement, puis corrigés — et c'est précisément leur rôle. Le premier : les réponses d'erreur de l'API arrivaient au navigateur sans en-têtes CORS, rendant les scénarios d'échec de connexion indéchiffrables côté front ; c'est la situation de recherche racontée en 18.3, résolue par l'ordre des middlewares. Le second : la consommation exactement égale au stock disponible était refusée à tort à cause d'une comparaison stricte — le cas limite a été corrigé et ajouté définitivement au jeu d'essai. Les dix-huit scénarios sont rejoués intégralement avant chaque jalon de fin de phase : c'est la forme de non-régression praticable pour un projet mené seul, en attendant leur automatisation, inscrite aux axes d'amélioration.

19.4. Le jeu de données de test

Les scénarios s'exécutent sur un jeu de données construit pour couvrir à la fois les cas nominaux et les cas limites. Trois **utilisateurs** — un gestionnaire, un technicien, un responsable magasin — permettent de vérifier la traçabilité nominative des mouvements. Quatre **véhicules** couvrent les variantes d'identification : un utilitaire au nouveau format d'immatriculation, un véhicule ancien portant les deux formats, un engin non immatriculé doté d'un identifiant libre, et un véhicule à fort kilométrage destiné aux butées kilométriques proches. Six **pièces** couvrent le magasin : le filtre à huile du jeu d'essai avec ses deux lots à prix différents, une pièce au seuil minimum élevé qui maintient une alerte permanente, une pièce à stock nul pour les tests de refus, et des pièces

partageant catégorie ou marque pour exercer l'autocomplétion. Enfin, huit **opérations** et plus — avec et sans consommables, butées calendaires passée et future, butée kilométrique dépassée, double butée —, une **méta-opération** de vidange à trois composantes, et trois **notes** — libre, rattachée à une opération, rattachée à la méta-opération — complètent le tableau. L'ensemble est ressaisi via Swagger après chaque réinitialisation, à partir de la collection de requêtes conservée dans le projet : les tests partent toujours du même état connu, condition de toute comparaison honnête entre attendu et obtenu.

20. Déploiement

Le déploiement actuel repose entièrement sur Docker, avec un périmètre différent selon les briques. La **base de données** est la brique la plus aboutie : le fichier docker-compose à la racine du dépôt décrit le service PostgreSQL 16, son volume nommé qui fait survivre les données aux redémarrages, ses identifiants de développement et son port ; une seule commande suffit à la faire exister sur n'importe quel poste. L'**application web** dispose de son image de production : un Dockerfile multi-stage construit d'abord le front dans une image Node 22 — installation des dépendances aux versions exactes du fichier de verrouillage, puis build Angular —, et copie ensuite les seuls fichiers produits dans une image nginx accompagnée de sa configuration. L'image finale ne contient ni Node, ni les sources, ni les dépendances de build : uniquement des fichiers statiques et un serveur web — elle est petite, rapide à démarrer, et n'expose aucun outil inutile.

La configuration nginx embarquée règle les deux problèmes classiques d'une SPA en production. Le **roulage côté client** d'abord : toute URL inconnue du serveur renvoie la page d'index, si bien qu'un rechargement sur une route interne de l'application — la fiche d'un véhicule, par exemple — fonctionne au lieu de produire une erreur 404. Le **CORS** ensuite : les appels vers le chemin /api sont relayés vers le service de l'API, de sorte que, vu du navigateur, tout provient de la même origine — le problème de cross-origin disparaît en production sans une ligne de configuration côté API.

L'**API** s'exécute en développement via le SDK .NET, la migration EF Core créant le schéma au premier lancement. L'**application desktop** se packal en un installateur Windows NSIS par une commande d'electron-builder. L'**application mobile**, enfin, est générée par Gradle depuis Android Studio ; sur l'émulateur, l'API de la machine hôte est atteinte par l'adresse dédiée 10.0.2.2, configurée dans le build.

20.1. L'installation sur un poste vierge

La procédure complète, validée en conditions réelles puisque c'est celle que j'utilise pour rejouer le jeu d'essai, s'enchaîne ainsi. On clone d'abord le dépôt GitHub. On démarre ensuite la base par docker compose : le container PostgreSQL se lance avec son volume. On applique la migration EF Core — une commande du SDK .NET — qui crée les onze tables, puis on lance l'API : Swagger est alors disponible et permet de vérifier que tout répond. Côté front, une installation des dépendances aux versions verrouillées puis le serveur de développement Angular donnent l'application sur le port 4200 ; la variante desktop se lance par le script Electron de développement, qui attend que le front soit prêt avant d'ouvrir la fenêtre. Pour la production, la construction de l'image Docker du front et le packaging de l'installateur desktop sont chacun l'affaire d'une commande. Pour le mobile, l'ouverture du dossier dans Android Studio suffit : Gradle télécharge les dépendances verrouillées et l'application se déploie sur l'émulateur. Les prérequis se résument à Git, Docker Desktop, le SDK .NET 8, Node.js, et Android Studio pour la partie mobile.

20.2. Vers la production : ce qui changerait

L'environnement actuel est un environnement de développement et de démonstration ; le passage en production réelle, esquissé dans les axes d'amélioration, changerait principalement quatre choses. Les **secrets** d'abord : la clé JWT et le mot de passe de la base quitteraient les fichiers de configuration versionnés pour des variables d'environnement injectées au déploiement — le dépôt n'en contiendrait plus aucun. Le **transport** ensuite : nginx terminerait le TLS avec un certificat automatisé,

et les en-têtes de sécurité — HSTS notamment — accompagneraient chaque réponse. L'**exécution** : l'API rejoindrait le docker-compose aux côtés de la base et du front, avec des healthchecks pour ordonner les démarrages et l'application automatique des migrations au lancement — la pile entière tiendrait dans une commande sur le serveur cible. La **supervision**, enfin : des journaux structurés et des métriques de containers, pour savoir ce qui se passe sans se connecter aux machines.

Aucun de ces changements ne touche au code applicatif : l'architecture actuelle — configuration externalisée, API sans état, images autonomes — a été pensée pour que la mise en production soit une affaire d'infrastructure, pas de réécriture. C'est, en creux, la validation du choix Docker fait au premier jour.

21. Axes d'amélioration

Le projet couvre le cahier des charges, mais un produit n'est jamais terminé — et savoir nommer ce qui manque fait partie du travail de conception. Les évolutions identifiées se regroupent en trois familles : les fonctionnalités, la sécurité et l'optimisation, et le déploiement.

21.1. L'ajout de fonctionnalités

La première évolution naturelle est l'**édition de rapports**, le bonus du sujet que je n'ai pas retenu : générer un document PDF depuis le journal d'un véhicule, avec l'en-tête réglementaire — attributs du véhicule, période et plage de kilométrage couvertes —, les opérations sélectionnées et leurs remarques, les notes libres, et la synthèse des opérations récurrentes avec leurs échéances. Toutes les données nécessaires sont déjà en base ; il ne manque que la mise en forme. Viendrait ensuite la **gestion des rôles et permissions** — distinguer technicien, responsable magasin et administrateur, en portant les rôles dans les claims du jeton et en les vérifiant par attributs d'autorisation —, indispensable avant tout déploiement multi-équipes réel.

Trois compléments d'usage suivraient. La **reconnaissance avancée des véhicules** sur mobile : lecture de plaque par la caméra, ou tags NFC, en complément du QR code. Le **mode hors-ligne** : une file d'attente locale des saisies, synchronisée au retour du réseau, pour les ateliers sans couverture — le chantier le plus lourd de cette liste, car la synchronisation soulève des questions de conflits qui méritent une conception à part entière. Et la **recherche avancée** : filtrer le journal par période, type d'opération ou consommable utilisé, avec la pagination des listes qui l'accompagne naturellement.

21.2. La sécurité et l'optimisation

Côté sécurité, quatre chantiers sont identifiés par ordre de priorité. Les **refresh tokens** d'abord : remplacer la déconnexion sèche au bout de deux heures par un renouvellement silencieux — jeton d'accès court, jeton de rafraîchissement révocable — qui améliore à la fois l'expérience et la capacité de révocation. La **gestion des secrets** ensuite : sortir la clé JWT et les mots de passe de la configuration versionnée, au profit de variables d'environnement ou d'un coffre dédié. Le **HTTPS de bout en bout** avec les en-têtes de sécurité — HSTS, CSP — sur nginx. Et la **limitation de débit** sur l'endpoint de connexion, que le middleware de rate limiting de .NET 8 rend simple à poser, contre la force brute.

Côté qualité et performances, le chantier prioritaire est celui des **tests automatisés** : les services métier — le stock et les butées s'y prêtent parfaitement, puisqu'ils sont isolés derrière des interfaces — appellent des tests unitaires ; l'API, des tests d'intégration sur une base conteneurisée éphémère ; le front, des tests de composants. Les **optimisations** de données suivraient à mesure du volume : projections EF Core pour ne charger que les colonnes utiles, index complémentaires sur les clés étrangères les plus sollicitées, et mise en cache des listes d'autocomplétion, qui changent rarement.

21.3. Le déploiement

Le premier pas est le **docker-compose complet** : ajouter les services de l'API et du front au fichier existant — la configuration nginx anticipe déjà un service nommé api — pour lancer toute la pile en une commande, avec des healthchecks et l'application automatique des migrations au démarrage du container de l'API. Vient ensuite la **chaîne d'intégration et de déploiement continu** : un workflow

GitHub Actions qui construit et teste à chaque push, fabrique les images Docker et les publie sur Docker Hub, puis déploie par docker compose sur le serveur cible. L'**observabilité** compléterait l'ensemble : journaux structurés, métriques, supervision des containers.

21.4. La feuille de route priorisée

Si le projet se poursuivait, j'engagerais ces chantiers dans un ordre précis, en équilibrant la valeur et l'effort. D'abord les tests automatisés des services métier et le docker-compose complet : un effort faible qui sécurise tout le reste — le FIFO et les butées sont déjà testables tels quels, et la pile complète en une commande fiabilise chaque démonstration. Ensuite la chaîne CI/CD avec publication sur Docker Hub, prolongement naturel du premier chantier. Puis les refresh tokens et la gestion des secrets, qui préparent une vraie mise en production. La génération de rapports PDF viendrait en quatrième, pour sa valeur métier immédiate. Les rôles et permissions en cinquième, avant tout déploiement multi-équipes. Et le mode hors-ligne mobile en dernier — le plus lourd, réservé à une version ultérieure.

22. Conclusion

Hesias Fleet répond au cahier des charges du Major Project : une solution de gestion de flotte déclinée sur trois supports, adossée à une API REST unique, avec les règles métier exigées — magasin à déstockage FIFO par prix, journal des véhicules mêlant opérations, méta-opérations et notes, butées calendaires et kilométriques avec notification, saisie assistée, et traçabilité complète des mouvements de stock. L'application web s'exécute en container Docker comme demandé, l'application desktop réutilise intégralement le front Angular grâce à Electron, et l'application mobile Kotlin offre au technicien un outil de saisie rapide, jusqu'au scan du QR code d'un véhicule.

Sur le plan des compétences, ce projet m'a fait pratiquer l'ensemble du spectre du titre Concepteur Développeur d'Applications : la conception d'un modèle relationnel et la mise en place de PostgreSQL, la construction d'une application multicouche en C# et .NET 8, le développement front-end et back-end web sécurisés, le développement mobile natif, et la préparation d'un plan de tests avec son jeu d'essai. Les principales difficultés — la mise au point de la règle FIFO par lots, l'ordre des middlewares CORS et authentification, la cohérence d'un contrat REST servi à trois clients — ont été autant d'occasions de recherche et de montée en compétence, restituées dans ce dossier.

Les axes d'amélioration identifiés — rapports PDF, rôles, refresh tokens, tests automatisés, chaîne CI/CD et compose complet — dessinent la feuille de route naturelle du produit. La base est saine : une architecture en couches claire, un contrat d'API documenté, des données fiables et tracées. C'est exactement ce qu'il faut pour faire grandir Hesias Fleet.

Enfin, sur le plan personnel, ce projet marque une étape : c'est la première fois que je mène seul un produit de bout en bout — de la lecture d'un cahier des charges à un livrable installable sur trois supports, en passant par les maquettes, la conception d'une base, la sécurité et les tests. J'en retiens que la méthode compte autant que le code : ce sont le backlog priorisé, les phases nettes et le contrat d'API stabilisé tôt qui ont rendu le périmètre tenable. Ce sont ces réflexes, plus encore que les technologies elles-mêmes, que j'emporte pour la suite de mon parcours de concepteur développeur d'applications.

23. Glossaire

API REST. Interface de programmation exposée en HTTP, où les ressources sont identifiées par des URL et manipulées par les verbes GET, POST, PUT, PATCH et DELETE, avec des échanges en JSON. C'est le contrat unique que consomment les trois clients du projet.

Butée. Échéance à laquelle une opération d'entretien doit être renouvelée : calendrier — une date — et/ou kilométrique — un kilométrage cible. Une opération est « due » dès qu'une de ses butées est atteinte.

Clean architecture. Organisation du code en couches dont les dépendances pointent vers le domaine métier, lequel ne dépend d'aucune technologie. Dans le projet : Core ne dépend de rien, Infrastructure dépend de Core, l'API dépend des deux.

Consommable. Pièce du magasin utilisée lors d'une opération — filtre, plaquettes, huile — et décomptée du stock au moment de la saisie.

CORS. Cross-Origin Resource Sharing : le mécanisme par lequel un navigateur autorise, ou non, une page servie par une origine à appeler une API d'une autre origine. Traité en développement par une politique explicite, et supprimé en production par le proxy nginx.

DTO. Data Transfer Object : objet dédié aux échanges de l'API, distinct des entités de la base, qui définit exactement ce qui entre et ce qui sort du serveur.

FIFO par prix. Règle de déstockage du projet : les lots les moins chers d'une pièce sont consommés en premier, et à prix égal les plus anciens.

Guard et intercepteur. Deux mécanismes transversaux d'Angular : le guard s'exécute avant l'activation d'une route — ici pour exiger l'authentification —, l'intercepteur s'insère dans chaque requête HTTP — ici pour porter le jeton et traiter les réponses 401. L'application mobile applique les mêmes idées avec OkHttp.

JWT. JSON Web Token : jeton signé contenant l'identité de l'utilisateur, présenté à chaque requête dans l'en-tête Authorization, et validé par le serveur — signature, émetteur, audience, expiration — avant tout traitement.

Lot (StockEntry). Quantité d'une pièce entrée en stock à un prix de revient donné. Le stock disponible d'une pièce est la somme de ses lots restants, et la consommation FIFO opère lot par lot.

Méta-opération. Regroupement d'opérations saisies ensemble — la « Vidange moteur » et ses composantes — pouvant porter sa propre butée.

Migration (EF Core). Code généré depuis les entités C#, versionné avec le projet, qui crée ou fait évoluer le schéma de la base de manière reproductible et réversible.

MVVM. Model-View-ViewModel : le patron d'architecture de l'application Android, qui sépare l'affichage — les Activities — de l'état et de la logique — les ViewModels.

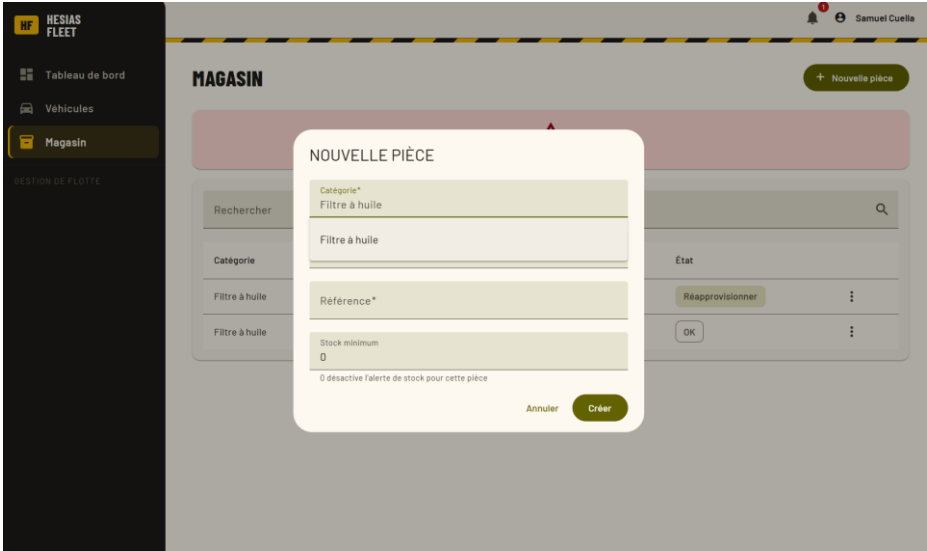
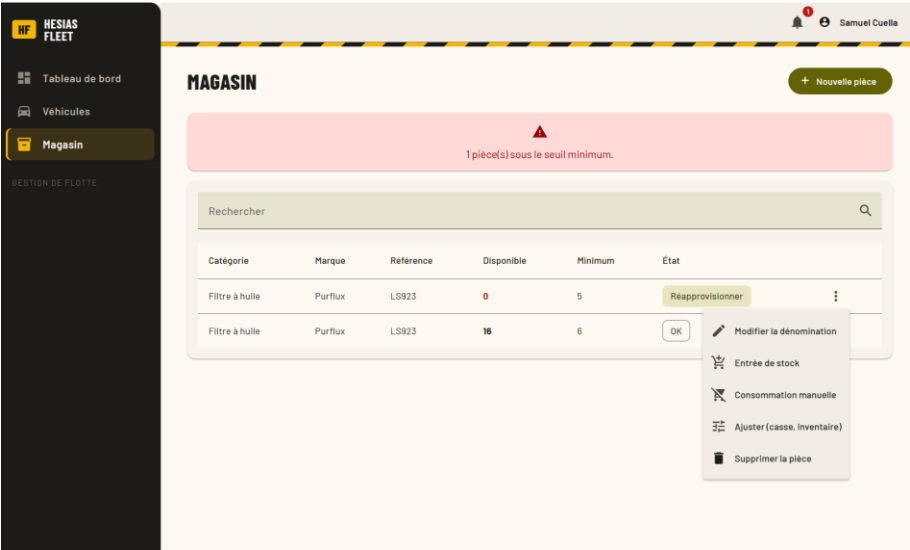
ORM. Object-Relational Mapping : l'outil — ici Entity Framework Core — qui fait le pont entre les objets du code et les tables relationnelles, et qui paramètre toutes les requêtes.

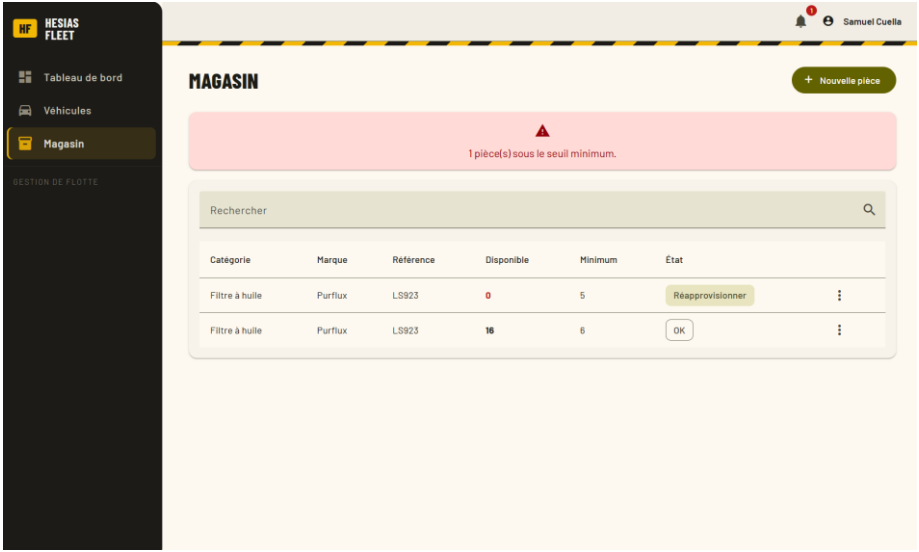
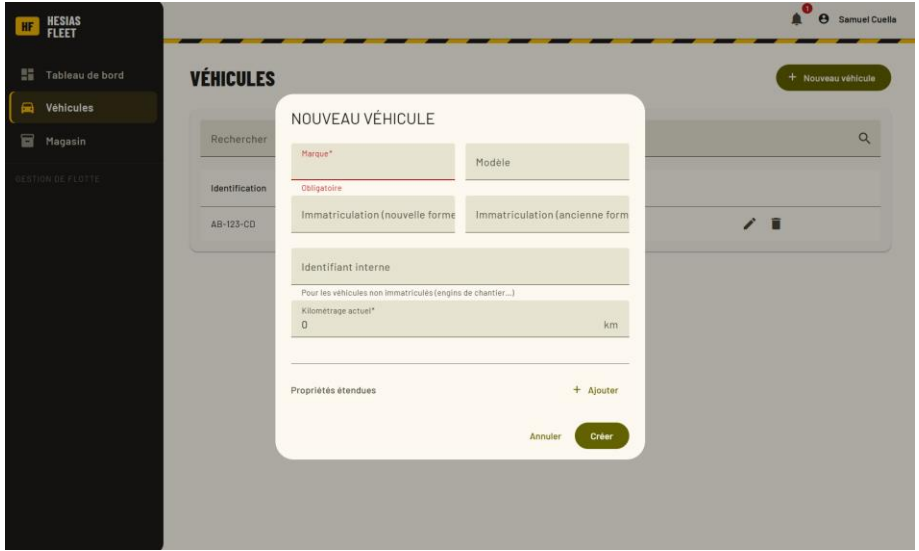
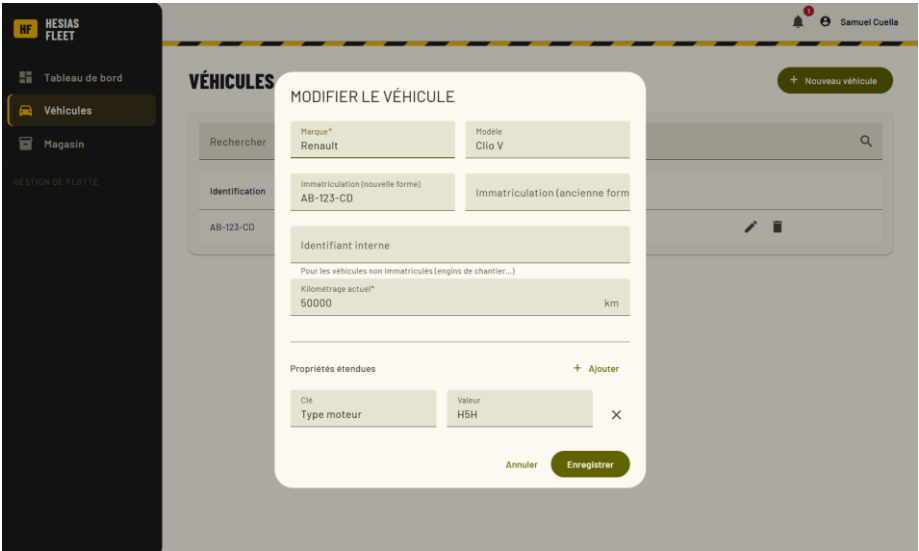
Signal. Primitive de réactivité d'Angular : une valeur observable dont la mise à jour redessine automatiquement les vues qui la lisent. Utilisée pour tout l'état partagé du front.

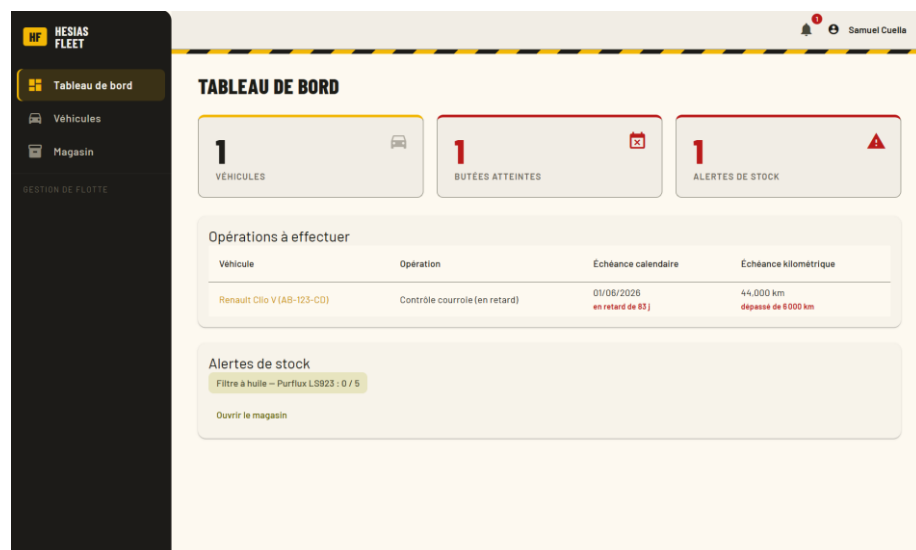
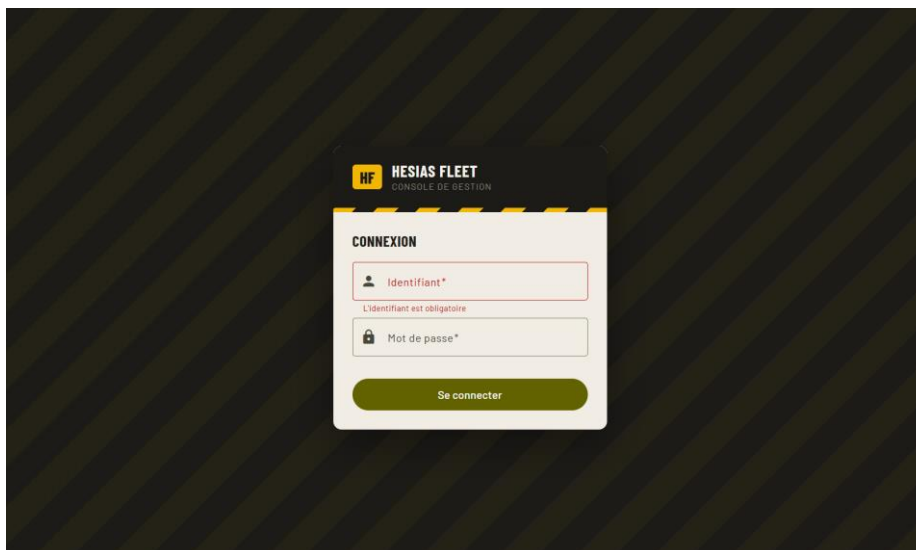
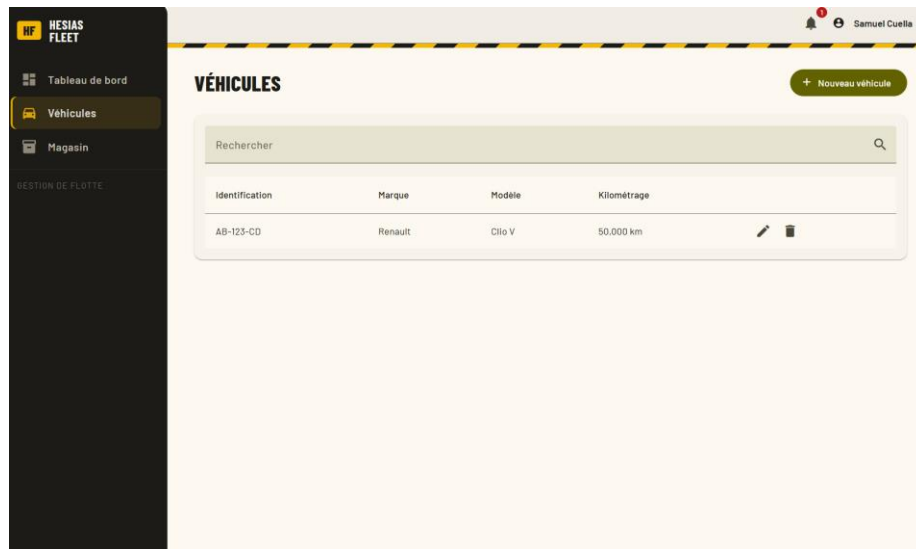
SPA. Single Page Application : application web chargée une fois, dont la navigation se fait côté client sans rechargement de page — d'où la règle nginx qui renvoie l'index pour toute URL inconnue.

Swagger / OpenAPI. Description normalisée de l'API, générée automatiquement, avec une interface web permettant de tester chaque endpoint — l'outil de travail de toute la phase backend.

Annexes :








```

1 0 > using
2
3 var builder = WebApplication.CreateBuilder(args);
4
5 // services
6 builder.Services.AddControllers();
7 builder.Services.AddEndpointsExplorer();
8 builder.Services.AddSwaggerGen(options =>
9 {
10 // definition of the JWT security scheme (Bearer)
11 options.AddSecurityDefinition("Bearer", new Microsoft.OpenApi.Models.OpenApiSecurityScheme
12 {
13     Name = "Authorization",
14     Type = Microsoft.OpenApi.Models.SecuritySchemeType.Http,
15     Scheme = "Bearer",
16     BearerFormat = "JWT",
17     In = Microsoft.OpenApi.Models.ParameterLocation.Header,
18     Description = "Enter the token JWT (same as prefix 'Bearer ').",
19 });
20
21 // apply the scheme to all protected endpoints
22 options.AddSecurityRequirement(new Microsoft.OpenApi.Models.OpenApiSecurityRequirement
23 {
24     { new Microsoft.OpenApi.Models.OpenApiSecurityScheme
25     {
26         Reference = new Microsoft.OpenApi.Models.OpenApiReference
27         {
28             Type = Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
29             Id = "Bearer"
30         },
31         Array.Empty<string>()
32     }
33 });
34
35 // database PostgreSQL via EF Core
36 builder.Services.AddDbContext<AppDbContext>(options =>
37 options.UseNpgsql(builder.Configuration.GetConnectionString("DB")));
38
39 // repositories (dependency injection)
40 builder.Services.AddScoped<IUserRepository, UserRepository>();
41 builder.Services.AddScoped<IVehicleRepository, VehicleRepository>();
42 builder.Services.AddScoped<IPartRepository, PartRepository>();
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

Auth

POST /api/Auth/login

[Try it out](#)

Parameters

No parameters

Request body

application/json

Example Value | Schema

```
{
  "login": "string",
  "password": "string"
}
```

Responses

Code	Description	Links
200	OK	No links

Media type: text/plain

Content Accept header

Example Value | Schema

```
{
  "token": "string",
  "user": {
    "id": "string",
    "lastname": "string",
    "firstname": "string",
    "function": "string",
    "email": "string",
    "login": "string"
  }
}
```

VehicleDto ▾ {

```

id > [...]
registrationOld > [...]
registrationNew > [...]
customIdentifier > [...]
brand > [...]
model > [...]
mileage > [...]
properties > [...]
}

```

VehiclePropertyDto ▾ {

```

key > [...]
value > [...]
}

```

WeatherForecast ▾ {

```

date > [...]
temperatureC > [...]
temperatureF > [...]
summary > [...]
}

```

```

StockConsumeDto {
  quantity      > [...]
}

```

```

StockEntryInputDto {
  quantity      > [...]
  unitCost      > [...]
}

```

```

StockStatusDto {
  partId        > [...]
  category      > [...]
  brand         > [...]
  reference     > [...]
  minimum       > [...]
  availableQuantity > [...]
  isBelowMinimum > [...]
}

```

```

UpdateMileageDto {
  mileage       > [...]
}

```

```

UserDto {
  id            > [...]
  lastName      > [...]
  firstName     > [...]
  function      > [...]
  email         > [...]
  login         > [...]
}

```

```

LoginDto {
  login         > [...]
  password      > [...]
}

```

```

MetaOperationDto {
  id            > [...]
  vehicleId     > [...]
  date          > [...]
  mileage       > [...]
  label         > [...]
  deadlineDate  > [...]
  deadlineMileage > [...]
  operations    > [...]
}

```

```

NoteDto {
  id            > [...]
  vehicleId     > [...]
  content       > [...]
  date         > [...]
  mileage       > [...]
  operationId   > [...]
  metaOperationId > [...]
}

```

```

NotificationDto {
  vehicleId     > [...]
  vehicleLabel  > [...]
  operationId   > [...]
  operationLabel > [...]
  deadlineDate  > [...]
  daysRemaining > [...]
  deadlineMileage > [...]
  kilometersRemaining > [...]
}

```

```

OperationConsumableDto {
  partId        > [...]
  quantity      > [...]
}

```

```

OperationDto {
  id            > [...]
  vehicleId     > [...]
  date          > [...]
  mileage       > [...]
  label         > [...]
  deadlineDate  > [...]
  deadlineMileage > [...]
  consumables   > [...]
  spareParts    > [...]
}

```

```

OperationSparePartDto {
  label         > [...]
  unitCost      > [...]
}

```

```

PartDto {
  id            > [...]
  category      > [...]
  brand         > [...]
  reference     > [...]
  minimum       > [...]
}

```

```

StockAdjustDto {
  newQuantity   > [...]
}

```

```

CreatePartDto {
  category > [...]
  brand > [...]
  reference > [...]
  minimum > [...]
}

CreateUserDto {
  lastName > [...]
  firstName > [...]
  function > [...]
  email > [...]
  login > [...]
  password > [...]
}

CreateVehicleDto {
  registrationId > [...]
  registrationDoc > [...]
  customIdentifier > [...]
  brand > [...]
  model > [...]
  mileage > [...]
  properties > [...]
}

DeadlineStatusDto {
  operationId > [...]
  label > [...]
  deadlineDate > [...]
  daysRemaining > [...]
  deadlineMileage > [...]
  kilometersRemaining > [...]
  idDue > [...]
}

```

Users

GET	/api/Users	🔒
POST	/api/Users	🔒
GET	/api/Users/{id}	🔒
DELETE	/api/Users/{id}	🔒

Vehicles

GET	/api/Vehicles	🔒
POST	/api/Vehicles	🔒
GET	/api/Vehicles/{id}	🔒
PUT	/api/Vehicles/{id}	🔒
DELETE	/api/Vehicles/{id}	🔒
PATCH	/api/Vehicles/{id}/mileage	🔒

WeatherForecast

GET	/WeatherForecast	🔒
-----	------------------	---

Parts

GET	/api/Parts	🔒
POST	/api/Parts	🔒
GET	/api/Parts/{id}	🔒
PUT	/api/Parts/{id}	🔒
DELETE	/api/Parts/{id}	🔒
GET	/api/Parts/categories	🔒
GET	/api/Parts/brands	🔒
GET	/api/Parts/references	🔒
GET	/api/Parts/{id}/stock	🔒
POST	/api/Parts/{id}/stock/entry	🔒
POST	/api/Parts/{id}/stock/consume	🔒
POST	/api/Parts/{id}/stock/adjust	🔒

Schemas

```

AuthResponseDto {
  token > [...]
  user > UserDto > [...]
}

CreateMetaOperationDto {
  vehicleId > [...]
  date > [...]
  mileage > [...]
  label > [...]
  headLineDate > [...]
  headLineMileage > [...]
  operations > [...]
}

CreateNoteDto {
  vehicleId > [...]
  content > [...]
  date > [...]
  mileage > [...]
  operationId > [...]
  metaOperationId > [...]
}

CreateOperationDto {
  vehicleId > [...]
  date > [...]
  mileage > [...]
  label > [...]
  headLineDate > [...]
  headLineMileage > [...]
  consumables > [...]
  spareParts > [...]
}

```

Auth

POST /api/Auth/login

MetaOperations

GET /api/MetaOperations

POST /api/MetaOperations

GET /api/MetaOperations/vehicle/{vehicleId}

GET /api/MetaOperations/{id}

DELETE /api/MetaOperations/{id}

Swagger

Select a definition HesiasFleetAPI v1

HesiasFleet.API 1.0 OAS 3.0

http://localhost:5131/swagger/v1/swagger.json

Authorize

Stack

- Kotlin, Views XML + ViewBinding
- Architecture MVVM (ViewModel + LiveData + coroutines)
- Retrofit 2 + Gson pour l'accès REST
- OkHttp Interceptor pour l'injection du JWT (miroir de l'authInterceptor Angular)
- EncryptedSharedPreferences pour le stockage chiffré du token
- ZXing (journeyapps) pour le scan QR code

6:18

Saisir une opération
Renault Clio V (AB-123-CD)

Méta-opération (regroupe plusieurs opérations)

Date

16/07/2026

Kilométrage

46000

Libellé (ex : Remplacement filtre à huile)

Pièces libres (saisies au vol)

+ Ajouter une pièce

Définir une butée (renouvellement)

Enregistrer

6:17

Renault Clio V (AB-123-CD)
50,000 km

Butée : à 76,000 km

Opération

Vidange - remplacement filtre
16/07/2026 - 46,000 km
Consommables : pièce #2 x 3
Pièces libres : Joint de vidange (2.5 €)
Butée : à 66,000 km

Opération

Remplacement filtre à huile
16/07/2026 - 45,000 km
Pièces libres : Joint de vidange (2.5 €)
Butée : à 66,000 km

Opération

Remplacement filtre à huile
16/07/2026 - 45,000 km
Pièces libres : Joint de vidange (2.5 €)
Butée : à 66,000 km

Opération

Contrôle courroie (en retard)
15/07/2026 - 40,000 km
Butée : le 01/08/2026 ou à 44,000 km

Saisir

6:17

Renault Clio V (AB-123-CD)
50,000 km

Butée

test 123
30/07/2026 - 46,500 km

Butée

Rayure sur l'aile avant droite constatée
16/07/2026 - 45,000 km

Méta-opération

Vidange moteur
16/07/2026 - 46,000 km
• Vidange huile usagée
• Remplacement filtre à huile
Butée : à 76,000 km

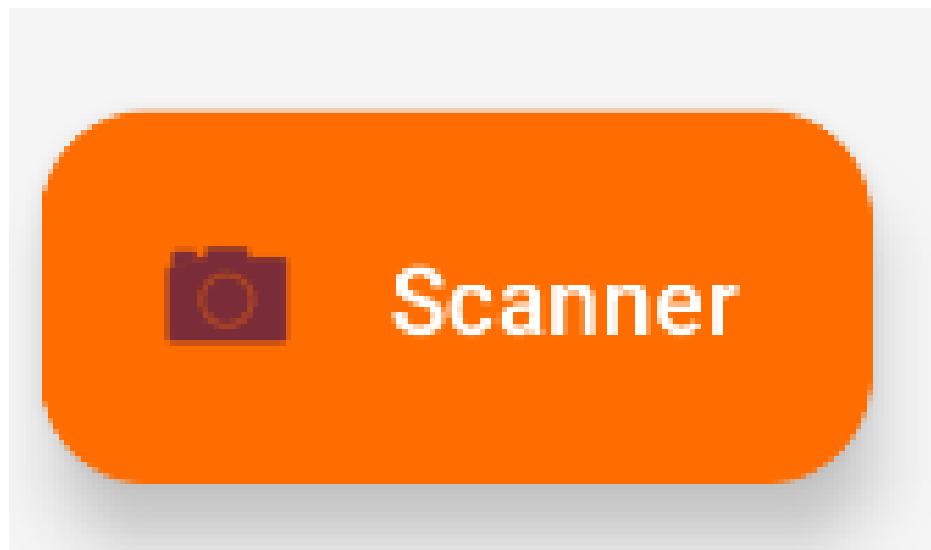
Opération

Vidange - remplacement filtre
16/07/2026 - 46,000 km
Consommables : pièce #2 x 3
Pièces libres : Joint de vidange (2.5 €)
Butée : à 66,000 km

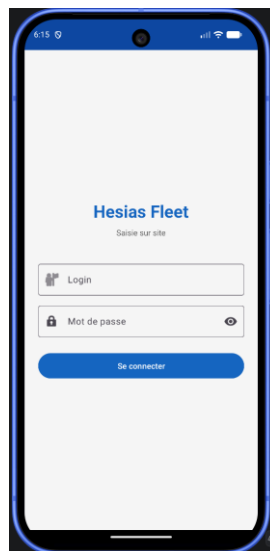
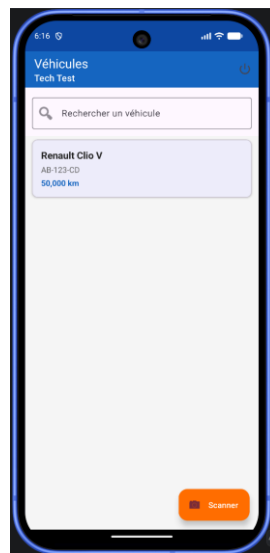
Opération

Remplacement filtre à huile
16/07/2026 - 45,000 km
Pièces libres : Joint de vidange (2.5 €)
Butée : à 66,000 km

Saisir



Véhicules Tech Test



Notes			^
GET	/api/Notes/vehicle/{vehicleId}		🔒 ▼
GET	/api/Notes/{id}		🔒 ▼
DELETE	/api/Notes/{id}		🔒 ▼
POST	/api/Notes		🔒 ▼
Notifications			^
GET	/api/Notifications		🔒 ▼
Operations			^
GET	/api/Operations		🔒 ▼
POST	/api/Operations		🔒 ▼
GET	/api/Operations/vehicle/{vehicleId}		🔒 ▼
GET	/api/Operations/vehicle/{vehicleId}/deadlines		🔒 ▼
GET	/api/Operations/{id}		🔒 ▼
DELETE	/api/Operations/{id}		🔒 ▼

